Traffic Simulator Using MPI

Project Report

SIT315 Concurrent and Distributed Programming
Task M4.T1D

Report contents

Sections 1 to 6 explain the problem, input, program design and new algorithm. Sections 7 to 12 explain commands, tests, measurements, results and stress outcomes. Sections 13 to 15 cover limitations, conclusions, and assistance. The evidence appendix includes every supplied screenshot in time order.

Topic	Main evidence
Correctness	Figures 1, 12 and 13
Machine specifications	Figures 2 and 12
Performance medians	Figures 3, 4 and 15
Candidate reduction and stage timings	Figures 11, 16, 17 and 18
Stress outcomes	Figures 5 to 9, 19 and 20
Prepared terminal demonstration	Figures 12 to 20

The figure numbers follow screenshot capture time. Repeated views are retained so the record is complete. All screenshots keep their original aspect ratio and are included without cropping.

Summary

This project extends a traffic simulator from SIT315 Tasks M2.T3D and M3.T3D. It reads traffic records and finds the top N busiest traffic lights for every hour. The final project keeps MPI and producer and consumer threads then adds binary message encoding, distributed top N selection, stage timings and stronger correctness tests. The aim is to reduce final collection work and investigate whether the changes improve performance.

The tests used an Apple M4 Mac with 10 cores, 10 logical threads, 16 GiB RAM and macOS 26.1. Three workloads contained 120000, 1200000 and 6000000 records. Each measured configuration ran three times. The improved MPI version had lower median times than the original MPI version in all six process and workload comparisons. The reductions in time ranged from 4.57 percent to 8.68 percent. However neither MPI version was faster than the sequential version on this machine.

For the large workload with four processes distributed selection reduced the entries collected at rank zero from 500000 complete keys to 6000 candidates. This is a 98.8 percent reduction in entries. A separate 60 million record run completed in 52.847 seconds of wall time. A 120 million record run did not finish within a 60 second limit and was stopped after 60.045 seconds. The results show a useful improvement to the existing MPI design while also showing that reading and local processing remain important limits.

1 Introduction

Traffic monitoring can produce many records. A traffic light may report a count every five minutes. If a system has many lights and covers many hours, it must read, group, and add millions of measurements. This project studies how concurrent and distributed programming can perform that analysis.

The input has three fields: timestamp, traffic light ID, and number of cars. The program groups measurements by hour and traffic light, adds their car counts, and prints the busiest lights for each hour. In this project, a higher car count is treated as a higher traffic volume. It is only a congestion indicator. The program does not measure waiting time, road capacity, vehicle speed, or queue length.

The earlier MPI version already produced correct results, but it added message conversion and final collection work. It converted summaries into text, exchanged them, and collected all complete totals at rank zero. The final project investigates whether a different message format and a smaller final candidate set can improve this design.

The project is batch analysis. It processes a file and prints hourly results after computation. A timestamp describes simulated time. The program does not wait five real minutes between measurements and does not control physical traffic lights.

Project aims

The aim is to improve the existing MPI traffic analysis program while keeping its hourly ranking results correct. The main question is whether binary encoding and distributed candidate selection reduce the median time of the original MPI implementation. A second question is how many entries need to be collected at rank zero. A third question is which measured stage takes the most time after the changes.

The evaluation also asks whether MPI beats the sequential baseline on one Mac. This question must be answered from measurements. A program can improve over its previous MPI version while still being slower than sequential processing. Both comparisons are needed to give a fair conclusion.

What the project includes

The implemented scope includes a reproducible data generator, the existing sequential reference, the original MPI baseline and the improved MPI program. It also includes correctness tests, repeated benchmarks, machine specification capture and time limited stress tests. Every MPI process uses the same input path on the same Mac.

The project does not implement a real sensor network a live dashboard, GPU processing, dynamic load balancing, fault tolerant recovery or communication and computation overlap. These are possible future changes. They are not claimed as features of this implementation.

2 Previous work and new changes

Task M2.T3D introduced sequential and multithreaded processing with a bounded producer and consumer queue. Task M3.T3D extended that work with MPI processes, byte range input splitting, local combining, hash ownership and distributed reduction. These are the base of this final project. They are existing work, not the new contribution.

The final project adds binary encoding for summary messages selection of top N candidates at each owner after complete totals are formed, communication counters, stage timings and an independent full total check. The uploaded original MPI source remains available as traffic_mpi_original.cpp for comparison. The improved implementation is traffic_mpi_optimised.cpp.

Feature	M2	M3	M4
Sequential reference	Yes	Retained	Retained
Producer and consumer threads	Yes	Within ranks	Retained
MPI and hash reduction	No	Yes	Retained
Binary summaries	No	No	New
Owner top N candidates	No	No	New
Stage and byte counters	No	No	New
Independent full total oracle	No	No	New

The M3 report identifies a difference between its implementation and the earlier task instruction about maintaining a sorted list during consumption. The MPI consumers update maps while ranking happens after reduction. The M4 project retains this batch ranking design and explains it openly. The M4 sheet asks for a new contribution and evaluation, it does not separately require a sorted list update after every record. This final report does not claim that the new project retroactively resolves every M3 instruction.

Sources used

The Open MPI documentation explains how MPI_Alltoallv handles different message sizes for different destinations. That supports the variable length exchange used here. The documentation for MPI_Init_thread explains the requested and provided thread support levels. The implementation requests MPI_THREAD_FUNNELED and checks that the provided level is sufficient.

MPI_Wtime provides elapsed wall clock time in seconds. The program measures a local duration on each rank and reports the maximum duration. It does not subtract timestamps from different ranks. The C++ working draft describes partial_sort, which selects and orders a leading part of a range. It also describes condition variable waiting with a predicate. Those features support candidate selection and the blocking bounded queue.

The distributed top N argument is developed for this project in Section 6. It relies on complete totals and a consistent tie rule. The binary format is also a project design choice. It uses explicit field lengths and a fixed byte order rather than copying a C++ object directly into a message. These research sources explain the tools, they do not prove that this specific program is faster. Performance claims come from the project measurements.

3 Input data

Data format

The required header is timestamp,traffic_light_id,cars. An example record is 2026-01-01 00:00:00,TL000001,27. It means that traffic light TL000001 recorded 27 cars at the stated time. A second record at 00:05 belongs to the same hourly group as the record at 00:00.

The program converts the timestamp to an hour label such as 2026-01-01 00:00. The key contains both that hour and the light ID. The value is a nonnegative car count. Counts for the same key are added. The final ranking uses total cars in descending order. If totals are equal, the light ID in ascending order decides the order.

There are 12 measurements per light per hour because 60 divided by 5 is 12. The number of generated records is lights multiplied by hours multiplied by 12. The header is not counted as a data record.

How the data is created

The generator starts at 1 January 2026. It produces valid dates across that year using month lengths. It uses the rule cars = 10 + ((17 multiplied by light number + 13 multiplied by hour index + 7 multiplied by slot index) modulo 91). Slot index runs from 0 to 11. The result is a count from 10 to 100.

This formula is deterministic. The same settings produce the same records which makes comparisons repeatable. It also produces repeated patterns and many equal totals. Those ties are useful for checking the ranking rule but the input is not a model of all real traffic patterns. It may make key distributions more regular than a real dataset.

The generator accepts at most 8760 hours and 1000000 lights. A requested 10000 hour stress dataset was rejected before MPI execution. The corrected test used 2000 lights and 5000 hours. This stayed within the limits and created the intended 120 million records.

Workload	Lights	Hours	Data records
Demo	5	2	120
Small	1000	10	120000
Medium	1000	100	1200000
Large	1000	500	6000000
Completed stress	1000	5000	60000000
DNF stress	2000	5000	120000000

Small example

The demonstration has five lights over two hours. It therefore contains 5 multiplied by 2 multiplied by 12 = 120 records. With two processes each process read 60 records in the observed run. The following table gives the expected top three totals.

Hour	First	Second	Third
00:00	TL000002 717	TL000001 695	TL000004 670
01:00	TL000002 691	TL000001 669	TL000005 666

A separate seven record file can be checked by hand. In the 08:00 hour TL01 has 10 + 25 = 35 cars. TL02 has 30 + 5 = 35 cars. TL03 has 20 cars. TL01 comes before TL02 because their totals are equal and TL01 has the smaller ID. In the 09:00 hour TL03 has 40 cars and TL01 has 7 cars. This example checks both addition and tie handling.

4 How the program works

Each MPI process has its own memory and its own rank number. Rank zero also reads and processes its input partition. It is not only a coordinator. Threads within one process share that process memory, but MPI ranks communicate through messages. Using both levels makes this a hybrid MPI and thread design inherited from M3.

The execution follows these steps. First MPI is initialised and arguments are checked. Second the input file is divided into byte ranges. Third producer threads read batches and place them in a bounded queue. Fourth consumer threads parse records and build private totals. Fifth the main thread combines private maps assigns keys to owner ranks and encodes messages. Sixth MPI exchanges the partial totals. Seventh each owner adds them to complete its keys. Eighth each owner selects its hourly top N candidates. Finally rank zero collects candidates and prints the global hourly top N.

How the file is divided

The program obtains the file size and skips the header. It divides the remaining bytes into approximately equal ranges. Equal bytes do not always mean equal numbers of records, because record lengths can differ. The data generator uses regular length records so the measured large run gave exactly 1500000 records to each of four ranks.

A boundary can fall inside a record. If a rank starts in the middle of a line, it skips the remainder. The previous rank finishes the line that began in its range. A producer checks the starting offset before reading a new line. This avoids intentionally splitting one record into two pieces or reading it twice. Empty ranges are allowed when there are fewer records than ranks.

The file is opened separately by each rank. Producer threads within a rank share one input stream protected by a file mutex. This allows separate MPI ranks to read different ranges but producer reads within a single rank are serialised. All ranks on the Mac still share the same storage system.

Threads and the queue

A producer reads a batch of raw lines. The batch has at most min(256, queue capacity) records. It puts that batch in a deque based queue. A consumer removes a batch parses each line and updates its private totals map. A deque supports adding at one end and removing at the other so it is suitable for first in first out batches.

Queue capacity is measured in records not in batches. With capacity 1024 queued batches cannot together hold more than 1024 records. Producers wait when there is not enough room for the whole batch. Consumers wait

when no batch is available. A mutex protects the queue, its record count and the closed and cancelled flags. Condition variables wake threads when data or space becomes available.

Waiting uses a predicate checked while the mutex is held. The predicate is checked again after waking. This is important because a wakeup does not guarantee that the required condition still holds. The file lock is released before waiting to enqueue and parsing happens outside the queue lock. These choices shorten shared critical sections.

A bounded queue limits queued records only. It does not limit the size of all maps, message buffers, current consumer batches, or pending producer batches. The program is therefore not a fixed memory streaming system for arbitrary input size.

Thread safety and stopping

Each consumer writes to a private map. Other consumers do not change that map. The main thread merges maps only after joining the workers. This avoids a shared map lock for every car count. The atomic producer count identifies the final producer which closes the queue. Consumers drain remaining batches and finish when the queue is closed and empty.

If a worker reports an error the first exception is stored under an error mutex and the queue is cancelled. Cancellation wakes waiting producers and consumers. Started threads are joined before an error is rethrown. If a rank reports an unrecoverable error, the program calls MPI_Abort. This stops the job instead of reporting successful partial output.

Only the main thread calls MPI. The workers finish before shuffle communication begins. The program uses blocking collectives. It does not use nonblocking MPI exchanges and does not overlap the main exchange with local computation. No deadlock was observed in the completed tests but those tests are not a proof for every possible execution.

Structure	Purpose	Protection
Deque of batches	Queued raw records	Mutex and condition variables
Input stream	Reads one rank range	File mutex within rank
Private maps	Hour and light totals	One consumer writer
Atomic producer count	Last producer closes queue	Atomic update
Exception pointer	First worker failure	Error mutex
Buckets and binary strings	MPI summary messages	Main thread only
Ranking vectors	Select and order candidates	Main thread only

5 MapReduce and sending data

Map and local totals

Map means turning a record into a key and a value. For this project the key is the hour and light ID and the value is the car count. Records for the same key are combined locally before communication. For example if one rank sees values 10, 20, and 5 for the same key it sends a partial total of 35 rather than three separate measurements.

Local combining can reduce the number of exchanged entries. Its benefit depends on how many repeats occur inside each input partition. If records for a key are scattered across ranks several partial totals for that key still need to be sent. Combining is already part of the original M3 design and is retained in M4.

How each key gets an owner

A stable hash is calculated from the hour and light ID. The owner is hash(key) modulo the number of ranks. Every occurrence of the same key therefore chooses the same owner. This gives one rank responsibility for the final total of that key. The hash is used for distribution not for security.

The program places partial totals into destination buckets. It first exchanges message lengths using MPI_Alltoall. It calculates counts and offsets then exchanges the encoded data using MPI_Alltoallv. The latter allows each destination to receive a different amount of data. A rank with no entries still participates with zero counts.

Hash distribution does not guarantee equal work. A rank can receive more unique keys or more partial contributions than another rank. The synthetic large run happened to assign 125000 complete keys to each rank. That observation applies to this run. It is not evidence of dynamic load balancing.

Reduce and collecting results

An owner decodes received entries and adds all partial values for each key. A complete total includes every rank contribution to that key. This is the Reduce step. Only after this step is it safe to use that key in candidate selection.

The final collection also uses the same exchange helper. Each rank puts its candidate payload in the bucket for rank zero and leaves the other buckets empty. It is logically a collection at rank zero but the implementation uses MPI_Alltoallv rather than MPI_Gatherv. Rank zero combines the candidates, groups them by hour, sorts them, and prints the first N lights.

The byte counters count encoded payload bytes submitted by all ranks. They include the rank zero self destination and do not measure physical network traffic, transport headers, count exchange traffic, or internal MPI copies. On one Mac, MPI can use local communication paths. It would be inaccurate to call these counters measured network bandwidth.

6 New changes

When the top N lights are selected

Selecting top N from partial totals can lose the correct winner. Consider N equal to one. On rank A light X has a partial total of 60 and light Y has 100. On rank B light X has 60 and light Z has 100. If both ranks keep only their best partial light they discard X. However X has a complete total of 120 and is the global winner. Therefore early selection is unsafe.

The improved program first sends all partial totals to their owner ranks. Each owner completes the sums. It then selects at most N lights per hour from its complete keys. Rank zero needs only those candidates to find the global result.

To explain why this is correct, take a complete light total excluded by its owner. There are already at least N lights in that hour on that owner that rank ahead of it. These N lights also rank ahead globally. Therefore the excluded light cannot be in the global top N. The reasoning uses the same total and ID tie rule at every stage. A key has only one owner so complete candidates are not duplicated across owners.

For H hours, P ranks, and top N equal to N, the candidate count is at most H multiplied by P multiplied by N. It can be lower if an owner has fewer than N keys for an hour. The bound concerns collection after reduction. It does not remove the need to shuffle partial totals first.

Selecting the top N lights

The selectCandidates function groups complete owner totals by hour and uses std::partial_sort to place the best N entries at the beginning. It orders cars from highest to lowest then IDs from smallest to largest. Partial sorting avoids fully ordering entries that will be discarded. The selected entries are placed in a candidate map for encoding.

For a large hourly group with K entries and a small N partial sorting generally requires work proportional to K log N comparisons rather than fully sorting K entries. This describes the selection algorithm not a measured whole program speedup. The function still stores all complete owner totals and makes temporary hourly vectors.

Binary message format

The original program formats totals as quoted text. The improved program encodes each entry as: eight bytes for the hour string length, the hour bytes, eight bytes for the light ID length, the ID bytes and eight bytes for the car total. Integers are stored with the most significant byte first. This is a fixed big endian order. The decoder reads the same layout and checks that fields fit within the remaining message.

Explicit lengths allow spaces in IDs and embedded zero bytes in the encoded representation without using a text line separator. Messages are kept in std::string with an explicit size and sent as MPI_CHAR elements. The program does not copy std::string objects, pointers, or padded C++ structs into the message. The representation avoids decimal formatting and quoted text parsing for the numeric total.

Binary encoding is not automatically smaller. In the measured generated data the hour label has 16 bytes and the light ID has eight bytes. Each binary entry therefore has 8 + 16 + 8 + 8 + 8 = 48 bytes. A typical complete text entry with a three digit total needs about 34 bytes in the original quoted format. The binary format can therefore be larger for these keys. The project claims lower conversion work as a design aim and a smaller candidate collection

as an observed result. It does not claim a measured reduction in shuffle bytes compared with the original program.

The implementation checks 64 bit sum overflow in the improved totals addition function. It also checks that message sizes and offsets fit the int interface used by MPI_Alltoallv. A very large job can still exceed those limits. Chunked exchanges or large count interfaces would be future work.

Checking all totals

The optional --all-totals argument disables candidate pruning and prints every complete total as a TOTAL record. The Python test reads these records and compares them with an independent dictionary calculation from the CSV. This checks keys that are not visible in the final top three output.

This diagnostic mode is used on small test files. It is not used in the performance benchmarks. In normal operation the program prints only the selected hourly rankings, counters, and timings.

7 Computer details and commands

Item	Observed value
Processor	Apple M4
Physical cores	10
Logical threads	10
RAM	17179869184 bytes = 16 GiB
Operating system	macOS 26.1 build 25B78
Architecture	arm64
MPI	Open MPI 5.0.10
Compiler	Apple Clang 17.0.0
Compiler target	arm64 apple darwin25.1.0

All performance runs used the same Mac and files within each workload. Open MPI processes ran on one machine. This is not a multi machine cluster experiment. The --bind-to none setting avoids restricting each rank to one core while it uses worker threads. The benchmark fixes one producer and one consumer per rank with capacity 1024. Four ranks therefore create eight worker threads in addition to the main threads although the main threads usually wait while workers process the input.

The following commands compile the programs with the same C++17 and O3 settings. The build script performs these commands and also enables warnings.

```
mpic++ -std=c++17 -O3 -Wall -Wextra -pthread \
  traffic_mpi_original.cpp -o traffic_mpi_original
mpic++ -std=c++17 -O3 -Wall -Wextra -pthread \
  traffic_mpi_optimised.cpp -o traffic_mpi_optimised
c++ -std=c++17 -O3 -Wall -Wextra -pthread \
  traffic_sim.cpp -o traffic_sim
c++ -std=c++17 -O3 -Wall -Wextra \
  generate_data.cpp -o generate_data
bash machine_specs.sh
python3 test_project.py
```

The settings after the input filename are top N, producers per rank, consumers per rank and buffer capacity in records. For example, the following command uses two MPI processes, top three, one producer and one consumer per rank, and capacity 1024.

```
head -n 8 data/traffic_demo.csv
./traffic_sim seq data/traffic_demo.csv 3
mpirun --bind-to none -np 2 ./traffic_mpi_original \
  data/traffic_demo.csv 3 1 1 1024
mpirun --bind-to none -np 2 ./traffic_mpi_optimised \
  data/traffic_demo.csv 3 1 1 1024
```

Each benchmark log records the exact command for its run. Replace np 2 with np 4 for four process tests and use traffic_small.csv, traffic_medium.csv, or traffic_large.csv for the relevant workload. The generator commands are shown below.

```
./generate_data data/traffic_small.csv 1000 10
./generate_data data/traffic_medium.csv 1000 100
./generate_data data/traffic_large.csv 1000 500
python3 benchmark.py
cat results/performance.csv
```

8 Correctness tests

The automated test uses five small datasets and three MPI process counts: 2, 4 and 8. For each dataset and process count it compares the original and improved MPI output with the sequential output. The signature includes hourly rankings, record count, and hour count. For the improved version, a separate full total diagnostic run is compared with the Python oracle.

There are 15 dataset and rank combinations. Each includes two normal MPI comparisons and one diagnostic MPI run. That gives 45 MPI runs in these small correctness checks. There are also five sequential references and one malformed input rejection test. The screenshots show all 15 PASS lines and the final ALL TESTS PASSED message.

Dataset	What it checks
Hand check	Known sums and equal total tie order
Demo	Two hours and 120 generated records
Uneven	1001 records for one key and five other keys
Empty	Header only and zero data records
Adversarial	Split winner A with total 24 and an ID containing spaces
Invalid	Malformed record must exit with failure

The malformed input test uses a timeout of 20 seconds. It requires a nonzero exit status. It shows that invalid data was rejected within the test limit. It does not prove every invalid input will be handled perfectly. Likewise passing empty and uneven tests gives evidence for those cases not a mathematical proof of the whole concurrent program.

The benchmark checks every measured output against its sequential reference. There are five configurations per workload: sequential, original MPI with two ranks, improved MPI with two ranks, original MPI with four ranks and improved MPI with four ranks. Three repetitions across three workloads give 45 measured runs. The script also obtains one extra sequential reference for each workload. Those reference runs are not included in the timing table.

The benchmark comparisons check printed rankings and record and hour counts. They do not independently compare every nonprinted light total in the large files. The full total check is limited to the small correctness files. This distinction keeps the correctness claim precise.

9 How performance was tested

The benchmark uses the program reported processing time rather than total command wall time. For MPI timing starts after initialisation and a starting barrier. It includes input reading, threads, parsing, aggregation, encoding, exchange, reduction, candidate selection, collection and final ranking. It excludes MPI launcher startup, input generation, final console printing and shutdown. MPI_Wtime is used for local elapsed durations.

The improved program also includes instrumentation collectives before recording its elapsed duration. The final time reduction itself is outside that local duration. Stage timings are maximum values across ranks. Stage maxima can come from different ranks so they must not be added to recreate total time.

The sequential timer includes its own reading, processing and sorting, but excludes printing. The previous sequential implementation maintains ordered sets during processing. The MPI consumers accumulate maps and sort later. Therefore comparison against sequential measures different complete implementations. It does not isolate the effect of MPI alone. The original versus improved MPI comparison is closer because the input processing design and settings are shared.

Every measured configuration runs three times. The median is the middle time after sorting the three values. Minimum and maximum show the range. Sample standard deviation describes variation between the three runs. Three observations are a small sample; no statistical significance test is claimed.

The benchmark runs configurations in a fixed order. It has no separate warmup protocol, randomised order, cold cache control, CPU temperature record, or background load record. These factors can influence small differences. The results should be understood as measurements from this test session.

Calculations

Speedup against sequential equals sequential median divided by the selected program median. A value above one means the program is faster than sequential. A value below one means it is slower. Speedup against original equals original MPI median divided by improved MPI median at the same process count.

Percentage reduction in time equals (original time minus improved time) divided by original time, multiplied by 100. For the large four process test: (5.411762 - 5.018556) divided by 5.411762, multiplied by 100 = 7.27 percent. The original comparison speedup is 5.411762 divided by 5.018556 = 1.07835.

Against sequential, 2.552227 divided by 5.018556 = 0.50856. Therefore the improved four process MPI program is about 1.97 times slower than sequential on this large workload. A 7.27 percent reduction from the original MPI time does not mean it beats sequential.

10 Performance results

Size	Version	Ranks	Median s	Vs seq	Time reduction vs old
Small	Sequential	1	0.041080	1.000	-
Small	Original	2	0.121814	0.337	-
Small	Improved	2	0.111245	0.369	8.68%
Small	Original	4	0.118760	0.346	-
Small	Improved	4	0.110098	0.373	7.29%
Medium	Sequential	1	0.455431	1.000	-
Medium	Original	2	1.312621	0.347	-
Medium	Improved	2	1.222910	0.372	6.83%
Medium	Original	4	1.103887	0.413	-
Medium	Improved	4	1.025222	0.444	7.13%
Large	Sequential	1	2.552227	1.000	-
Large	Original	2	6.590181	0.387	-
Large	Improved	2	6.288867	0.406	4.57%
Large	Original	4	5.411762	0.472	-
Large	Improved	4	5.018556	0.509	7.27%

The improved version has lower median time in every original versus improved comparison. The smallest reduction is 4.57 percent for the large two process case. The largest is 8.68 percent for the small two process case. Four process results show reductions of 7.29, 7.13, and 7.27 percent for small, medium and large workloads.

For the improved large workload moving from two to four processes reduces time from 6.288867 to 5.018556 seconds. That is a 1.25312 speedup relative to the improved two process version and a 20.20 percent reduction in time. However doubling processes does not halve runtime. File access, memory traffic, map work, coordination, and collective communication still contribute.

The sequential version is the fastest measured implementation at all three workload sizes. The observed large sequential time is 2.552227 seconds, compared with 5.018556 for improved MPI with four ranks. MPI is useful here as a distributed programming model and an optimisation study. These results do not support a claim that MPI improves absolute performance on this Mac.

The changes were applied together. There is no ablation experiment that measures binary encoding alone and candidate selection alone. Therefore the measured time reductions belong to the combined implementation. We cannot assign an exact fraction of the gain to either change.

Differences between repeated runs

Large version	Ranks	Minimum s	Maximum s	Sample SD s
Sequential	1	2.500148	2.576366	0.038953
Original	2	6.589910	6.591464	0.000828
Improved	2	6.142386	6.295577	0.086573
Original	4	5.405582	5.427702	0.011413
Improved	4	5.016866	5.053383	0.020613

The large two process improved runs have more spread than the original runs. The improved times range from 6.142386 to 6.295577 seconds while the original times range from 6.589910 to 6.591464 seconds. The large four process improved times range from 5.016866 to 5.053383 seconds. These values describe repeat variation within this session. More repeats and a randomised test order would improve confidence in small gains.

The small sequential median is 0.041080 seconds. Such a short duration is sensitive to measurement conditions. The medium and large workloads provide longer runtimes while the full set shows the increase in workload and time. The 120 record demonstration is for correctness not useful speedup analysis. Its measured time changed from 0.000948 to 0.001016 seconds across two demonstrations without changing its rankings.

11 Stage times and data counts

The detailed large four process log is run 1 with an execution time of 5.053383 seconds. This differs from the three run median of 5.018556 seconds. Stage analysis below uses that one detailed run and must not be presented as a median stage profile.

Measured stage	Maximum seconds	Included work
Map	4.950815	Reading parsing queues local totals and merging
Pack	0.047081	Hash buckets and binary summary encoding
Shuffle	0.052241	Count exchange payload exchange and buffers
Reduce	0.035240	Decode and add owner contributions
Candidates	0.008023	Select and encode owner top N
Collection	0.000365	Exchange candidates to rank zero
Ranking and metadata	0.001537	Count reduction rank gathers and final ranking

The map stage is the longest measured stage at 4.950815 seconds. It includes file setup, range reading, queue activity, parsing, private maps, thread completion and local map merging. It is not just the Map operation on an already loaded array. Its name covers the whole local input processing part.

The shuffle stage includes the count exchange message buffer assembly in the helper, the variable length collective and construction of the received string. It also includes waiting associated with those operations. A local elapsed stage is not a pure network transfer measurement. There are no barriers between all stages, so a rank can arrive early and wait for another rank.

The final collection time is 0.000365 seconds in this run. Candidate selection and encoding take 0.008023 seconds. The ranking and metadata stage includes record count reduction, rank statistics gathers, decoding at rank zero, and final candidate sorting. These details explain what the timers actually measure.

Counter	Observed value
Shuffle encoded bytes	24000000
Collection encoded bytes	288000
Complete keys	500000
Collected candidates	6000
Input records per rank	1500000 on all four ranks
Complete keys per rank	125000 on all four ranks

How fewer entries are collected

The large input has 1000 lights across 500 hours. It therefore has 1000 multiplied by 500 = 500000 complete keys. With four ranks and top three the candidate bound is 500 multiplied by 4 multiplied by 3 = 6000. The counter shows exactly 6000 collected candidates.

The removed entry count is 500000 - 6000 = 494000. The reduction percentage is 494000 divided by 500000, multiplied by 100 = 98.8 percent. The retained fraction is 6000 divided by 500000 = 0.012 or 1.2 percent. This is a reduction in final collected entries not a 98.8 percent reduction in total runtime or in all communication.

Every generated candidate has a 48 byte binary representation. The final payload is therefore 6000 multiplied by 48 = 288000 bytes matching the counter. Sending all 500000 complete keys in the same binary format would require 24000000 bytes. That is a calculated same format comparison not a measured original text collection size.

The shuffle counter is also 24000000 bytes in this run. It is a separate phase from final collection. It must not be used as if it were a direct measured old versus new collection comparison. The demo has only five lights per hour so its ten complete keys were all retained across the two owners. A small demo does not necessarily show candidate pruning.

Work per rank and the main limit

Each of the four ranks processed 1500000 records and owned 125000 complete keys. That is exact balance for this generated large file. Yet processing was still slower than sequential. Equal record counts do not remove queue handoff, private data structures, memory allocation, shared storage pressure and MPI coordination.

The very small final collection time and the large map duration suggest that further work should focus on local reading, parsing and aggregation. This is a measured direction for future optimisation. It does not establish the exact cause of local processing cost, because no lower level CPU, file I/O or memory profile was collected.

12 Stress tests

Completed stress test

The first stress dataset used 1000 lights and 5000 hours. It contained 1000 multiplied by 5000 multiplied by 12 = 60000000 records. The improved MPI program ran with four ranks, one producer and one consumer per rank, top three, and buffer capacity 1024.

The wrapper allowed 60 seconds. The command completed with exit code zero after 52.847 seconds of observed wall time. This is successful large workload execution evidence. It is not the required DNF evidence. The saved completed summary and output were copied before the next stress run.

Data setting error

An intermediate attempt requested 10000 hours. The generator rejected this because its maximum is 8760 hours. No new MPI run started for that attempt. The old stress_summary.txt still showed the successful 60 million record result, because failure happened before a new summary was written.

This is an input configuration error not an MPI timeout, compile failure or DNF of the analysis program. It is included in the evidence appendix to explain the visible traceback. The corrected setup increased the number of lights instead of exceeding the hour limit.

Stress test that did not finish

The corrected dataset used 2000 lights and 5000 hours: 2000 multiplied by 5000 multiplied by 12 = 120000000 records. The program used the same four rank settings and a 60 second processing limit. It did not finish within the limit. The wrapper terminated the process group and reported DNF caused by an imposed timeout.

The observed wall time was 60.045 seconds and the launcher exit code was 1 after termination. The extra 0.045 seconds includes wrapper and shutdown overhead. Exit code 1 here is interpreted together with the wrapper outcome. It is not by itself proof of a program crash. The input size does not mean all 120 million records were processed before termination.

Input records	Wall time s	Outcome	Exit code
60000000	52.847	Completed	0
120000000	60.045	DNF imposed timeout	1

The stress wall time includes launcher startup final output if produced, and process shutdown. Dataset generation is outside this timer. The normal benchmark internal timer excludes startup and printing. Therefore 52.847 seconds of stress wall time should not be directly treated as the same measurement as a benchmark internal duration.

The wrapper first sends a termination signal to the process group. If the processes do not exit within five seconds it sends a kill signal. This prevents MPI children from continuing indefinitely after the time limit. The evidence does not establish an out of memory event, a deadlock or the exact stage reached in the timed out run.

```
python3 stress_test.py
cp results/stress_summary.txt \
  results/stress_completed_60million.txt
cp results/stress_output.txt \
  results/stress_completed_60million_output.txt
python3 stress_test.py --lights 2000 --hours 5000 \
  --ranks 4 --timeout 60
cat results/stress_summary.txt
```

13 Limits and future improvements

The project runs all MPI processes on one Mac. It does not show scaling across physical computers or a real network. Storage bandwidth and memory resources are shared. Results on a cluster can differ. A future study should test multiple machines with clear storage and network specifications.

The data is synthetic and deterministic. It contains regular records and repeated count patterns. A future evaluation should include different record lengths, more realistic skew, and varied numbers of lights per hour. The existing uneven test checks correctness with skewed keys but it does not measure skewed workload performance.

Three benchmark runs are a small sample. The fixed order may favour later runs through caching or disadvantage them through temperature and background activity. There is no separate measurement of peak RAM, CPU usage, disk reads or network transfer. Timing explanations must remain proportional to this evidence.

Binary encoding carries explicit length fields and can be larger than short text records. Numeric IDs a shared dictionary for hours and IDs or narrower checked lengths could reduce payload size. Those changes would require careful validation and are not included in the measured program.

The program uses maps, copied strings, temporary buckets and temporary ranking vectors. The queue is bounded, but the summary maps grow with unique keys. Candidate selection reduces collection size; it does not bound owner memory. Chunked processing by hour or sorted input windows could improve memory use but would change the design and require further tests.

Input parsing checks the timestamp shape and time ranges but it does not validate every calendar date rule. It expects the exact header and simple unquoted input CSV fields. The improved addition function checks unsigned total overflow, while the old sequential baseline does not provide identical overflow checking. Tests use counts far below the limits.

The exchange uses blocking MPI_Alltoallv with int counts and offsets. It rejects payloads beyond its safe limits instead of supporting unlimited messages. Future work could use chunking or suitable large count MPI interfaces. Nonblocking communication would need a real plan for useful overlap; replacing a function name alone would not establish a benefit.

The strongest next experiment is an ablation study: original text with full collection binary with full collection, text with candidate collection and binary with candidate collection. This would help separate the effects of the two main changes. A faster parser and alternative aggregation structures could then target the dominant local processing stage.

14 Conclusion

The final project adds a clear contribution beyond the previous traffic simulator. It uses binary summaries, selects candidates after complete ownership reduction, records stage timings and byte counts and checks complete totals on small test datasets. The observed results remain correct for the tested hand check, demo, uneven, empty, and adversarial inputs.

The improved MPI version has lower median time than the original MPI version across all six measured comparisons. With six million records and four ranks its median is 5.018556 seconds rather than 5.411762 seconds, a 7.27 percent reduction. However sequential processing remains faster at 2.552227 seconds on the same workload.

The clearest structural result is the reduction from 500000 complete keys to 6000 collected candidates or 98.8 percent fewer entries. This does not imply the same reduction in total runtime. The stage profile shows that local input processing still takes most of the time in the detailed large run. The stress evaluation shows both successful execution at 60 million records and DNF at 120 million records under the same 60 second limit.

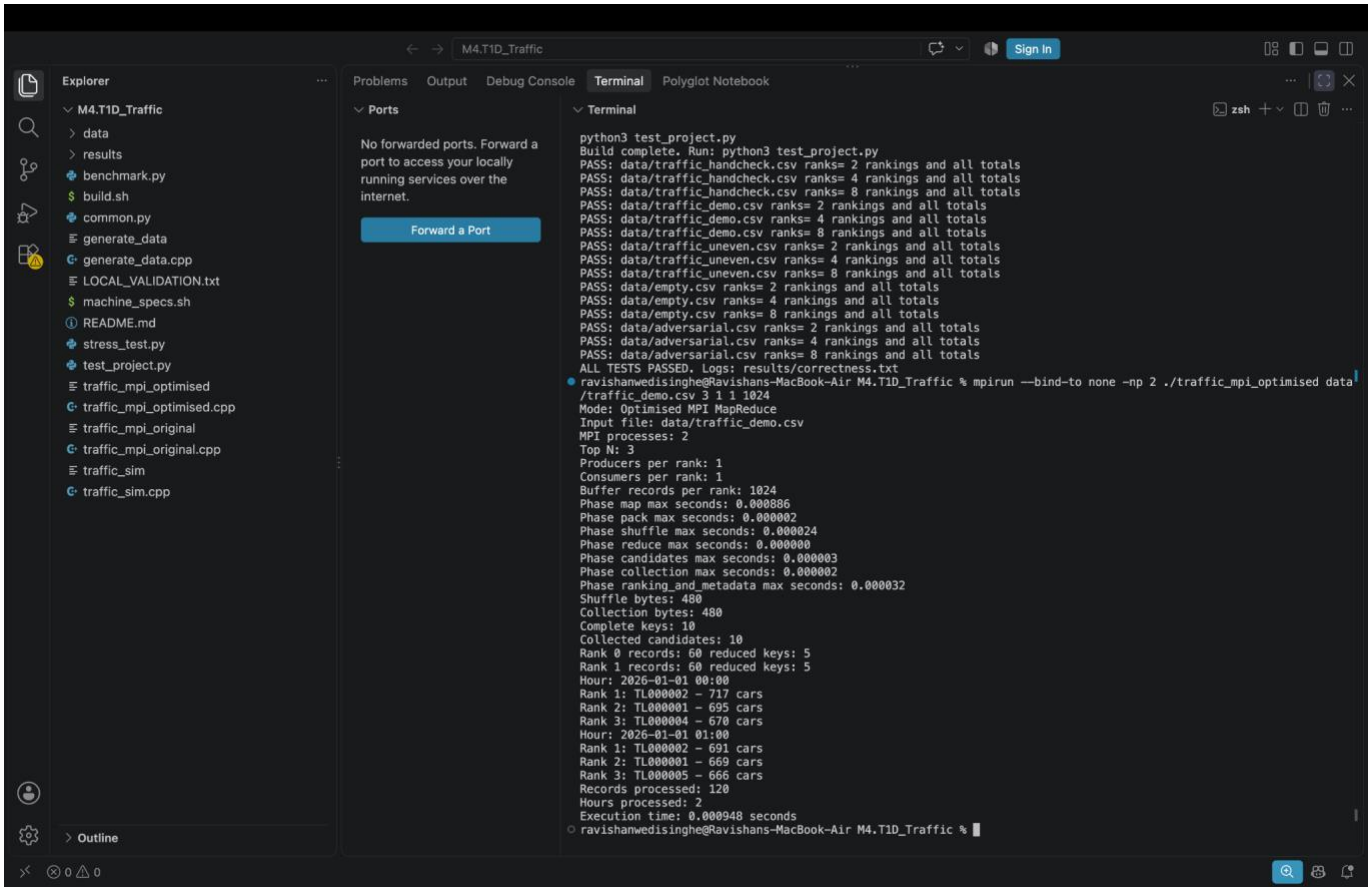
These findings support the combined improvement as a useful extension while setting clear limits on the performance claim. The project is ready to be explained through its code, commands, test outputs and measured evidence. A separate video demonstration must show the same features and code and satisfy the task sheet duration.

Screenshots

Every supplied screenshot is included below in capture time order. The original image is preserved without cropping. Captions explain the visible commands and outcomes. Several views repeat the same saved results; they are not counted as new benchmark runs.

Figure 1 Build tests and demo

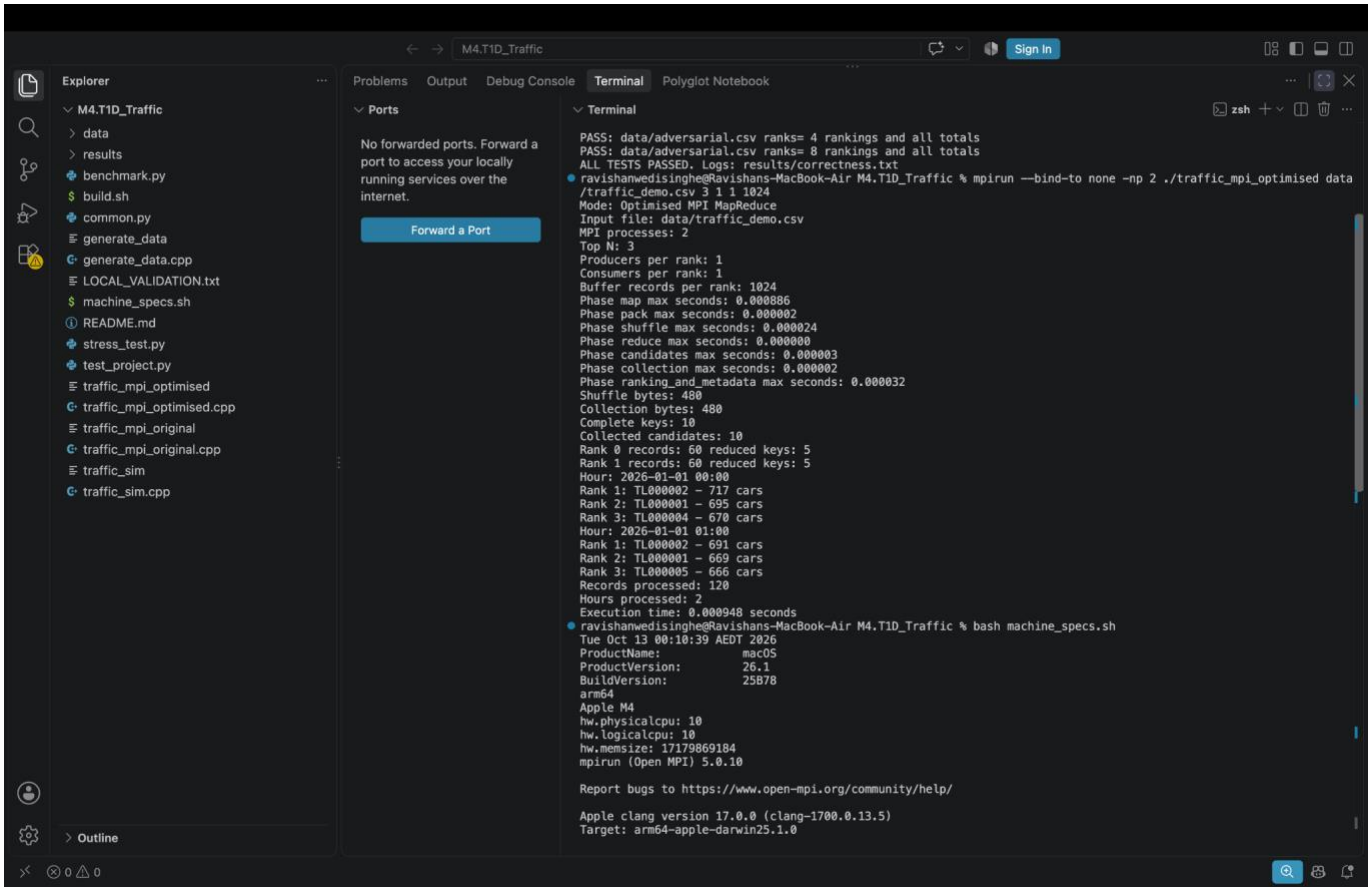
Screenshot captured on 13 October 2026 at 12:09:35 AM.



The build completed and all five correctness datasets passed with 2, 4, and 8 ranks. The improved demo ran with two ranks, top three, one producer and one consumer per rank, and capacity 1024. It processed 120 records across two hours. The 0.000948 second demo time is from this run only. The rankings match the expected demo table. Ten complete keys and ten candidates show that this small input does not prune entries.

Figure 2 Computer details

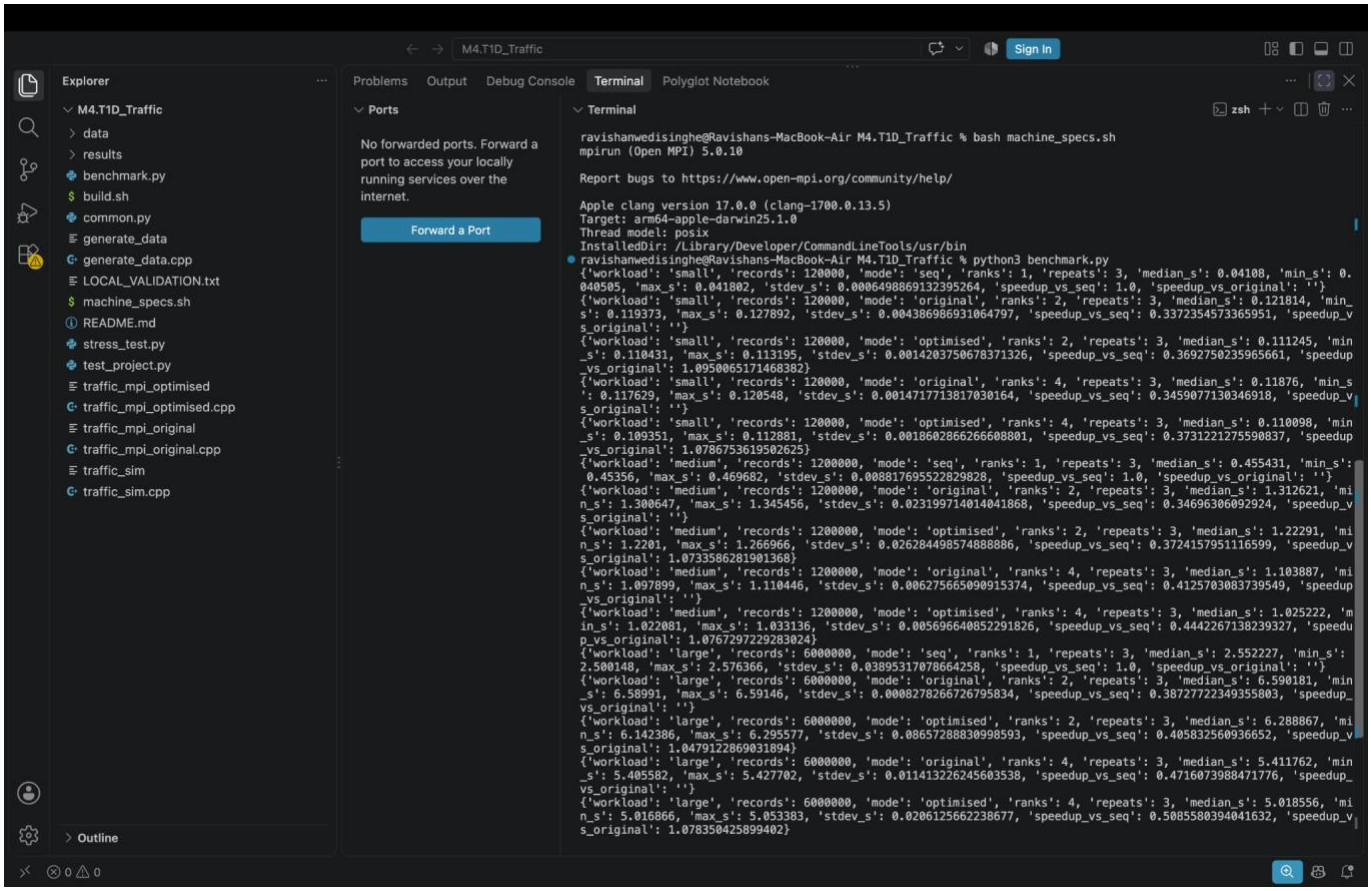
Screenshot captured on 13 October 2026 at 12:12:59 AM.



The terminal shows macOS 26.1 build 25B78, arm64, Apple M4, 10 physical cores, 10 logical threads, and 17179869184 bytes of RAM. That is 16 GiB. Open MPI is version 5.0.10. These details identify the tested machine and are needed when interpreting performance.

Figure 3 Repeated performance tests

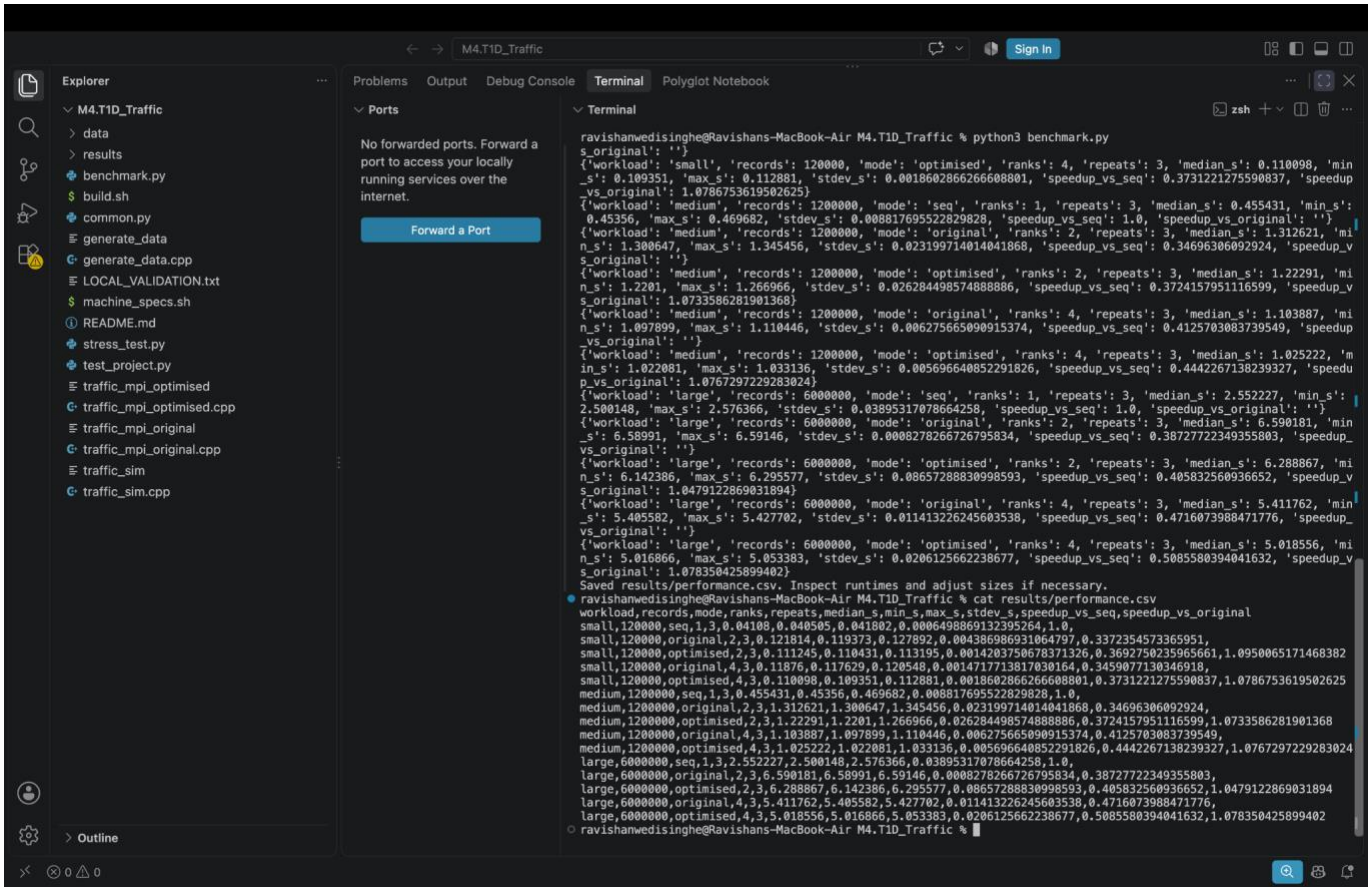
Screenshot captured on 13 October 2026 at 12:13:07 AM.



The benchmark prints sequential, original MPI, and improved MPI results for three increasing workloads. Each measured configuration has three repeats. Median, minimum, maximum, and sample standard deviation are shown. Speedup versus sequential is below one for the MPI runs. Speedup versus original is above one for the improved MPI runs. This means the improvement helps the MPI design but does not beat sequential processing.

Figure 4 Performance table

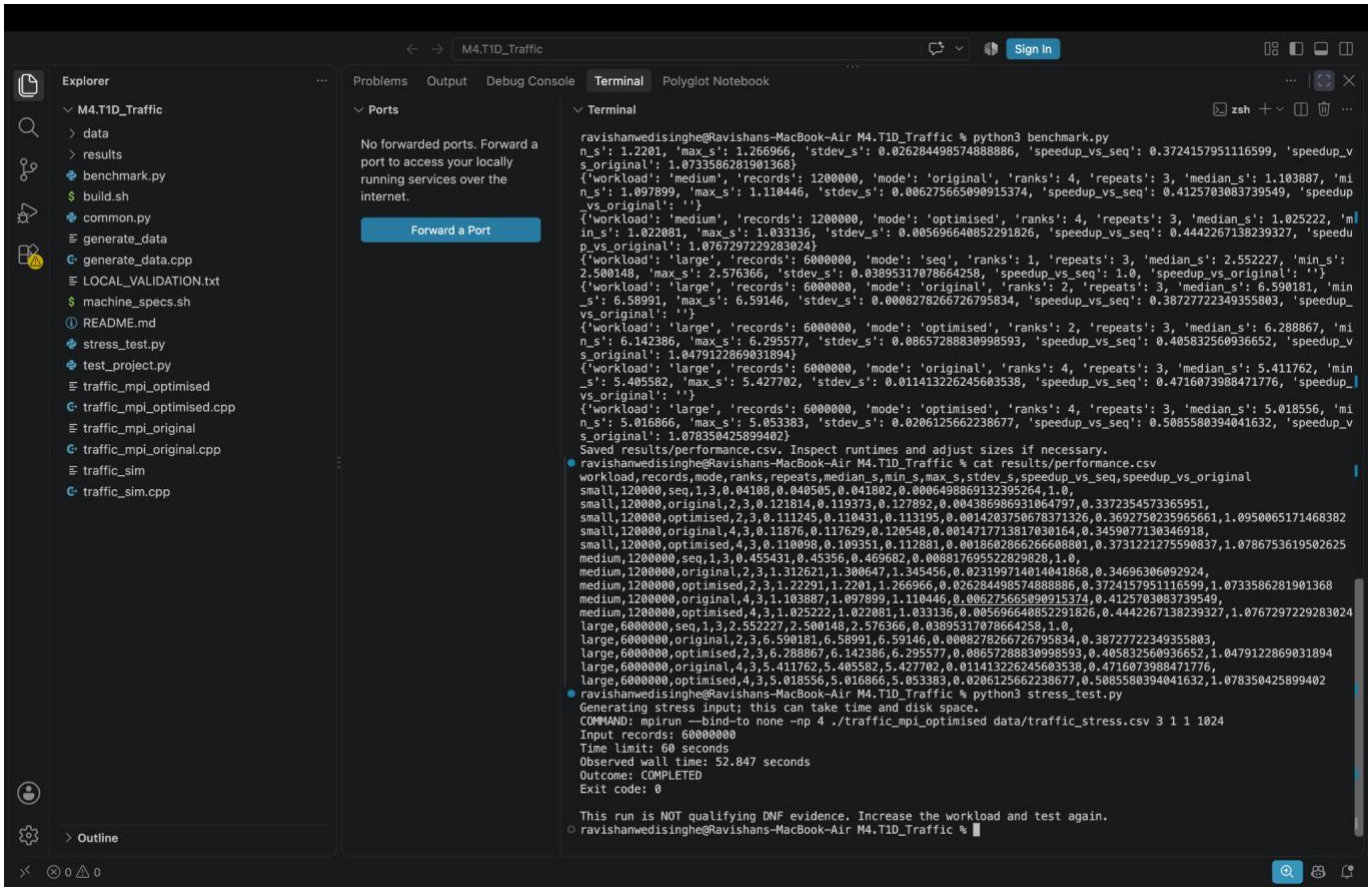
Screenshot captured on 13 October 2026 at 12:13:12 AM.



The final message confirms that performance.csv was saved. The displayed CSV contains all 15 configuration summaries. This is the source of the report median comparison table. Large improved MPI with four ranks has median 5.018556 seconds. Large sequential has median 2.552227 seconds. Printed benchmark comparisons completed without a reported mismatch.

Figure 5 Completed stress test

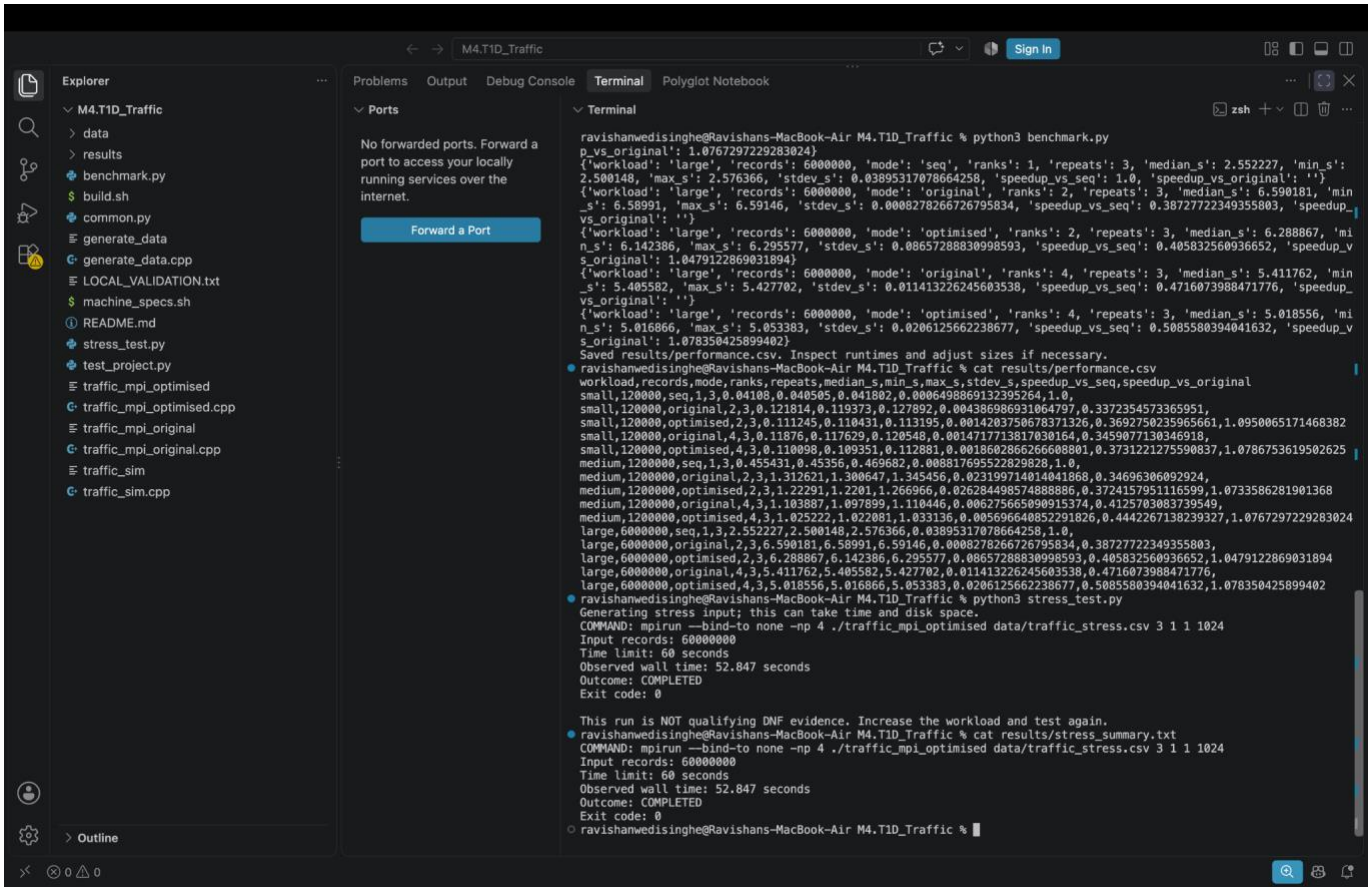
Screenshot captured on 13 October 2026 at 12:16:06 AM.



The stress input contains 60000000 records. Four MPI ranks ran with the usual one producer, one consumer, and capacity 1024. The wrapper reports 52.847 seconds of wall time, completed outcome, and exit code zero. The wrapper correctly states that this is not qualifying DNF evidence. It is successful large workload evidence.

Figure 6 Completed stress result

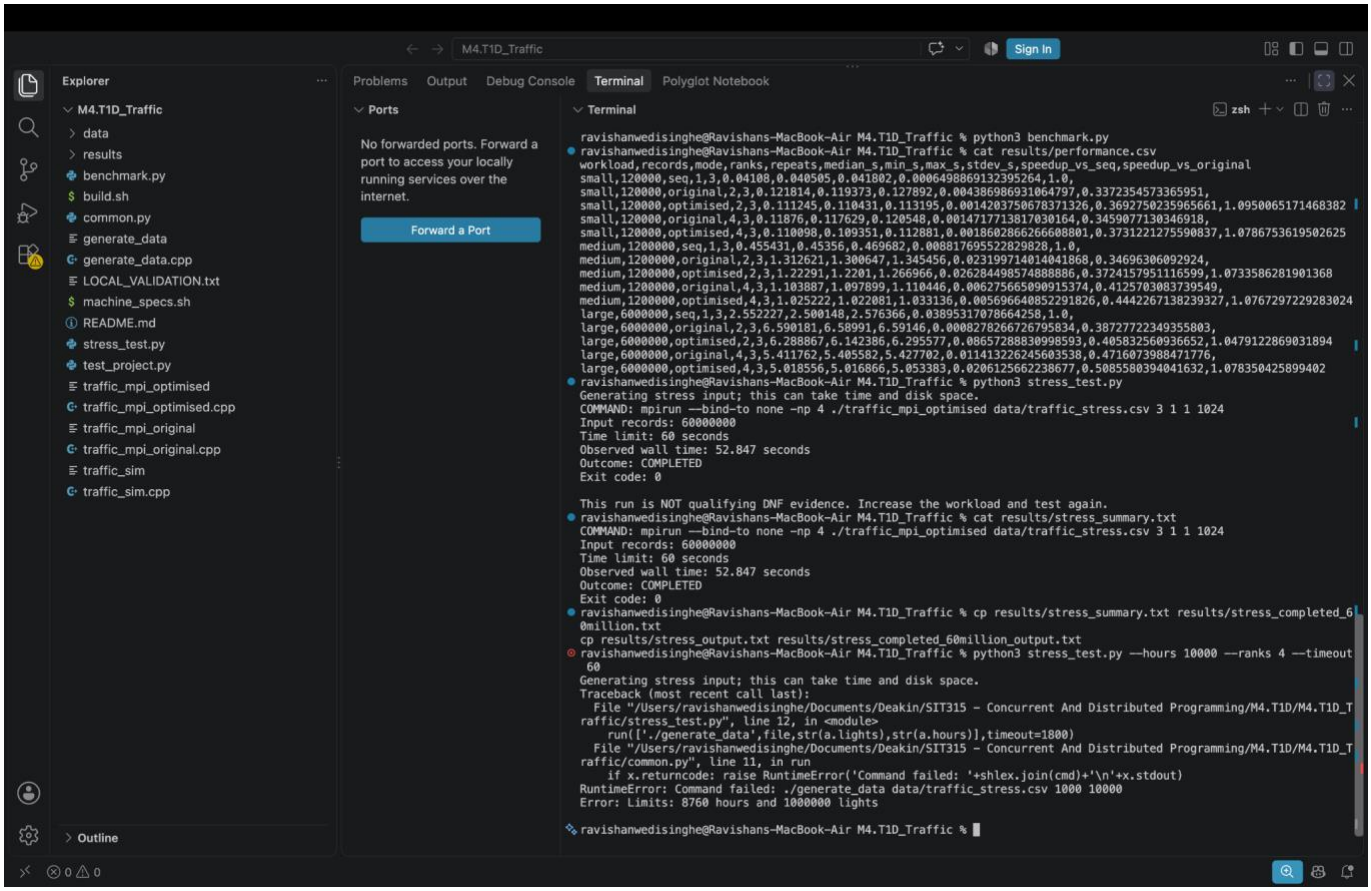
Screenshot captured on 13 October 2026 at 12:16:13 AM.



The saved summary repeats the exact MPI command, 60000000 record input size, 60 second limit, 52.847 second wall time, and successful exit. The successful summary was later copied to stress_completed_60million.txt. This screenshot confirms the stored outcome rather than starting another run.

Figure 7 Hour setting error

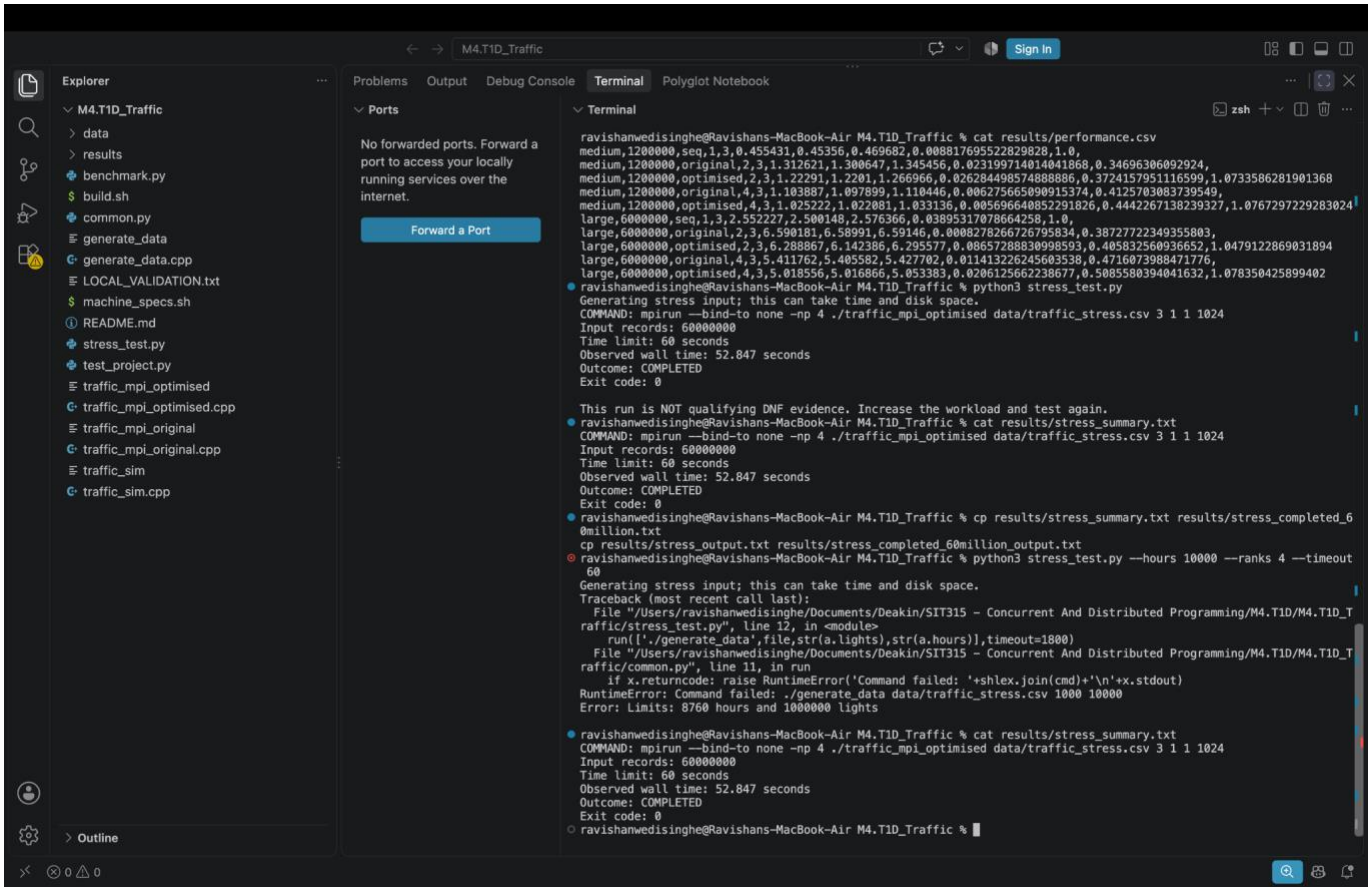
Screenshot captured on 13 October 2026 at 12:17:20 AM.



The successful stress logs were copied. The next attempt requested 10000 hours. The generator returned an error because its maximum is 8760 hours. The traceback is a generator setting failure before MPI execution. It is not a compilation failure or a timed out MPI workload. The test was corrected by using more lights within the supported hour range.

Figure 8 Previous result after the error

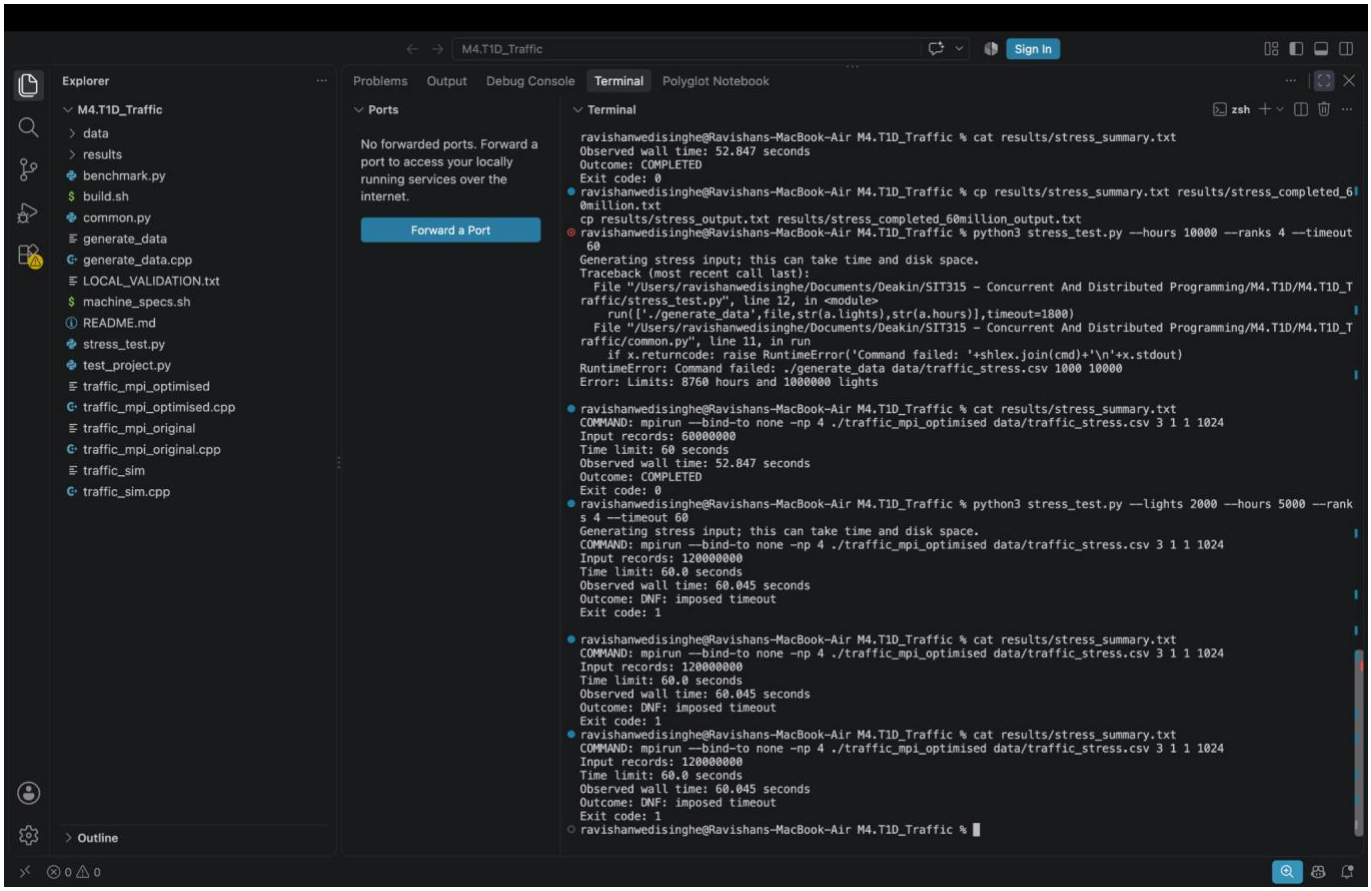
Screenshot captured on 13 October 2026 at 12:17:29 AM.



The generator rejection is visible above a displayed stress_summary.txt. That summary still describes the earlier 60000000 record successful run. The failed generation did not write a new summary or launch the intended MPI test. This prevents the old outcome from being mistaken for a result of the rejected settings.

Figure 9 Stress test stopped by timeout

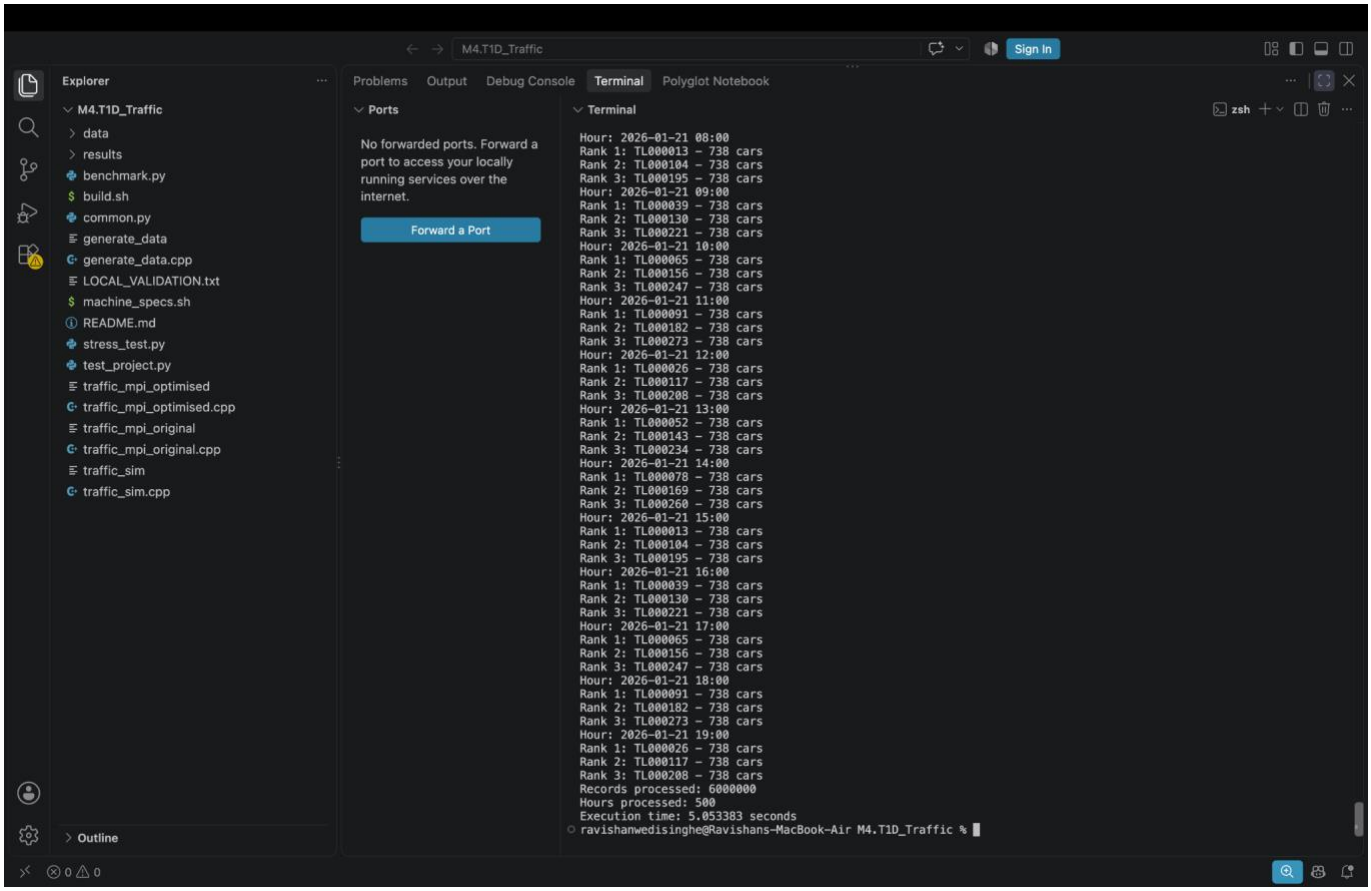
Screenshot captured on 13 October 2026 at 12:20:48 AM.



The corrected command uses 2000 lights and 5000 hours, giving 120000000 input records. Four ranks did not finish within 60 seconds. The wrapper reports DNF caused by an imposed timeout, observed wall time 60.045 seconds, and launcher exit code 1. Repeated cat commands display the same saved result, not additional stress runs.

Figure 10 Large test results

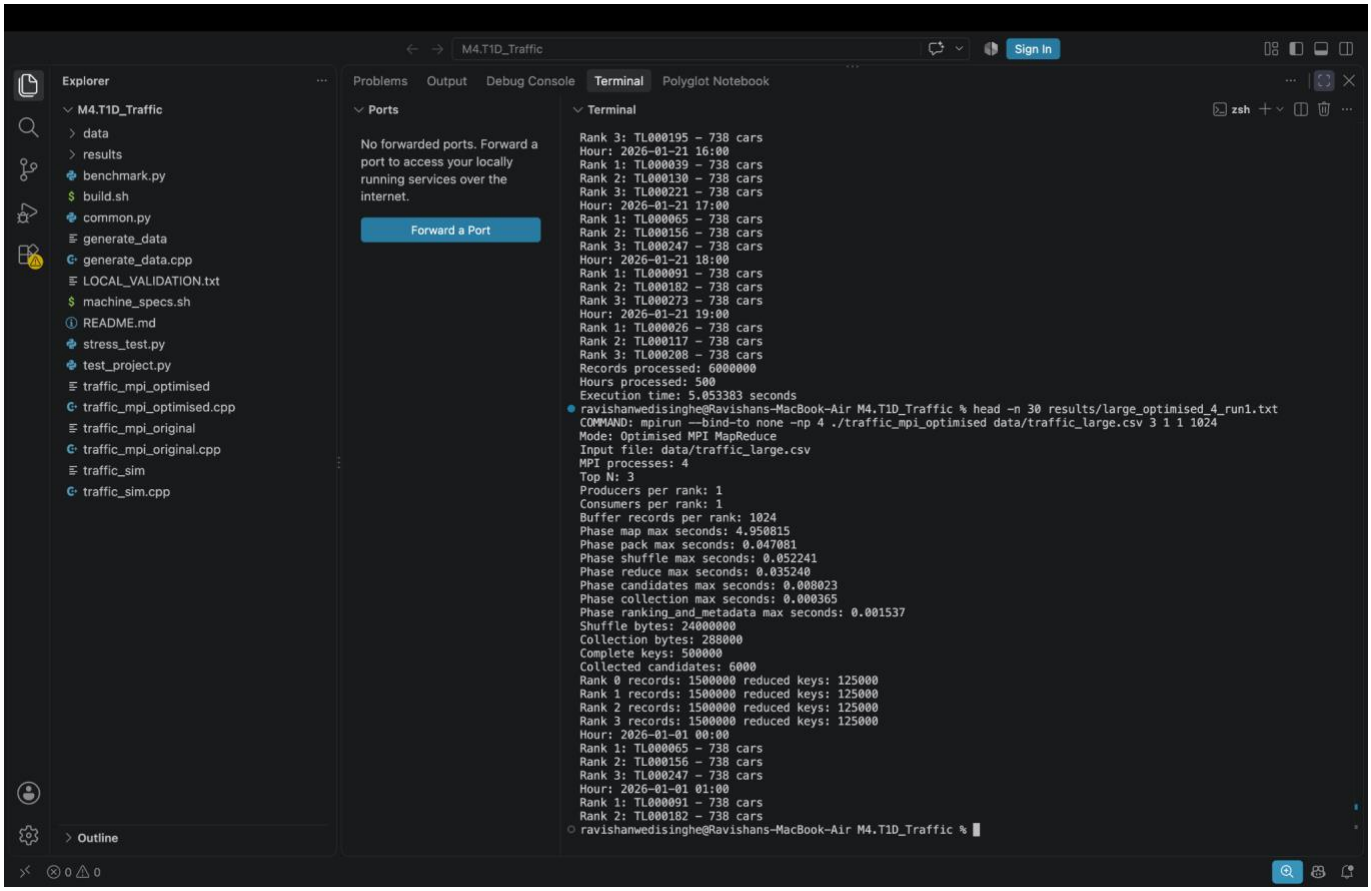
Screenshot captured on 13 October 2026 at 12:22:22 AM.



The terminal displays the end of the large run output. Hourly ties are ordered by traffic light ID. The run processed 6000000 records across 500 hours in 5.053383 seconds. This is run 1 from the large four rank benchmark. It is a single measured duration, not the three run median.

Figure 11 Stage times and fewer candidates

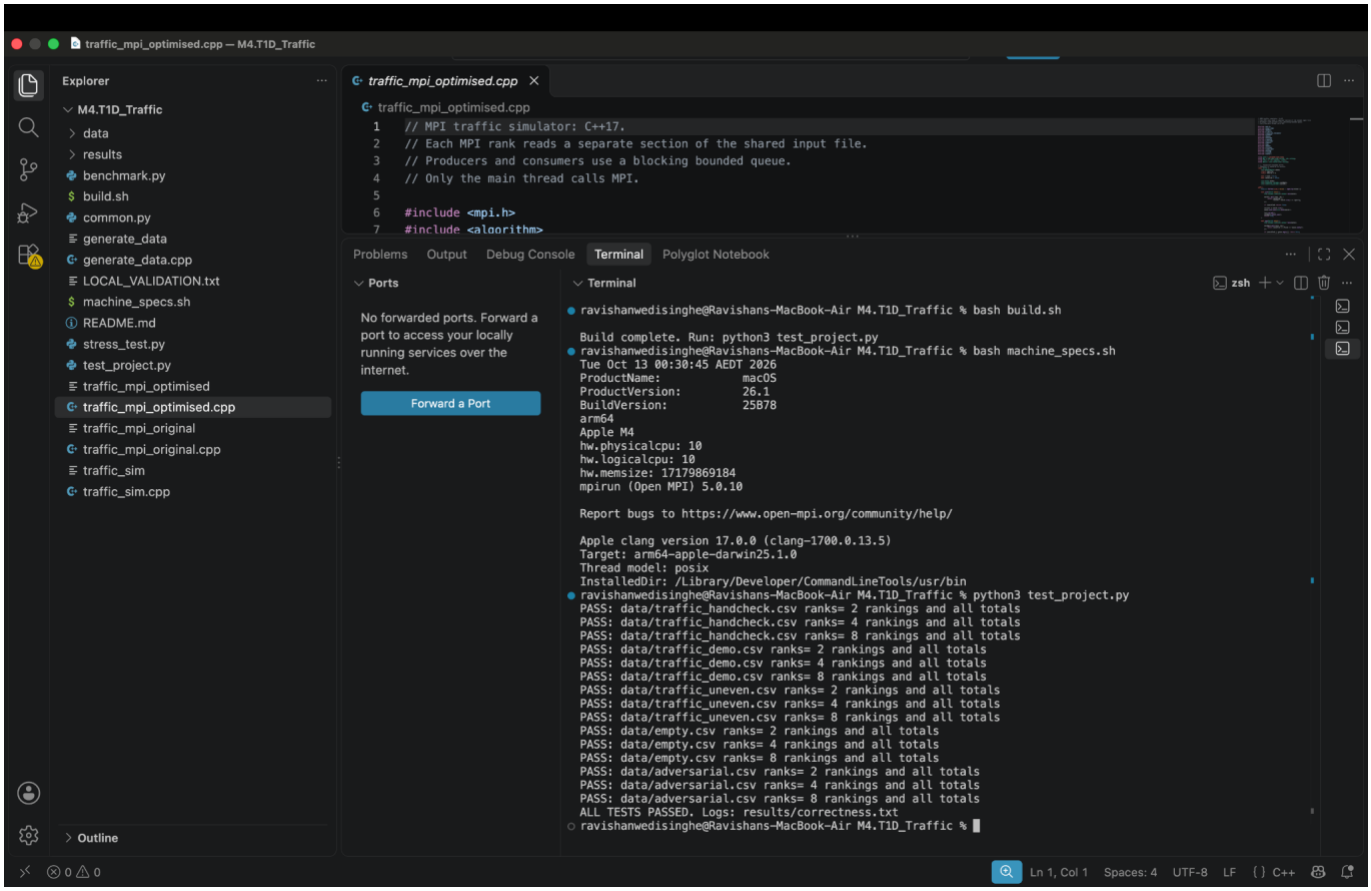
Screenshot captured on 13 October 2026 at 12:23:30 AM.



The first lines of the saved large run show all seven stage timers. Map is longest at 4.950815 seconds. Complete keys are 500000 and collected candidates are 6000, giving 98.8 percent fewer final entries. Collection payload is 288000 bytes. Each rank has 1500000 records and 125000 complete keys. Counters are encoded payload totals, not measured physical network traffic.

Figure 12 Build computer details and tests

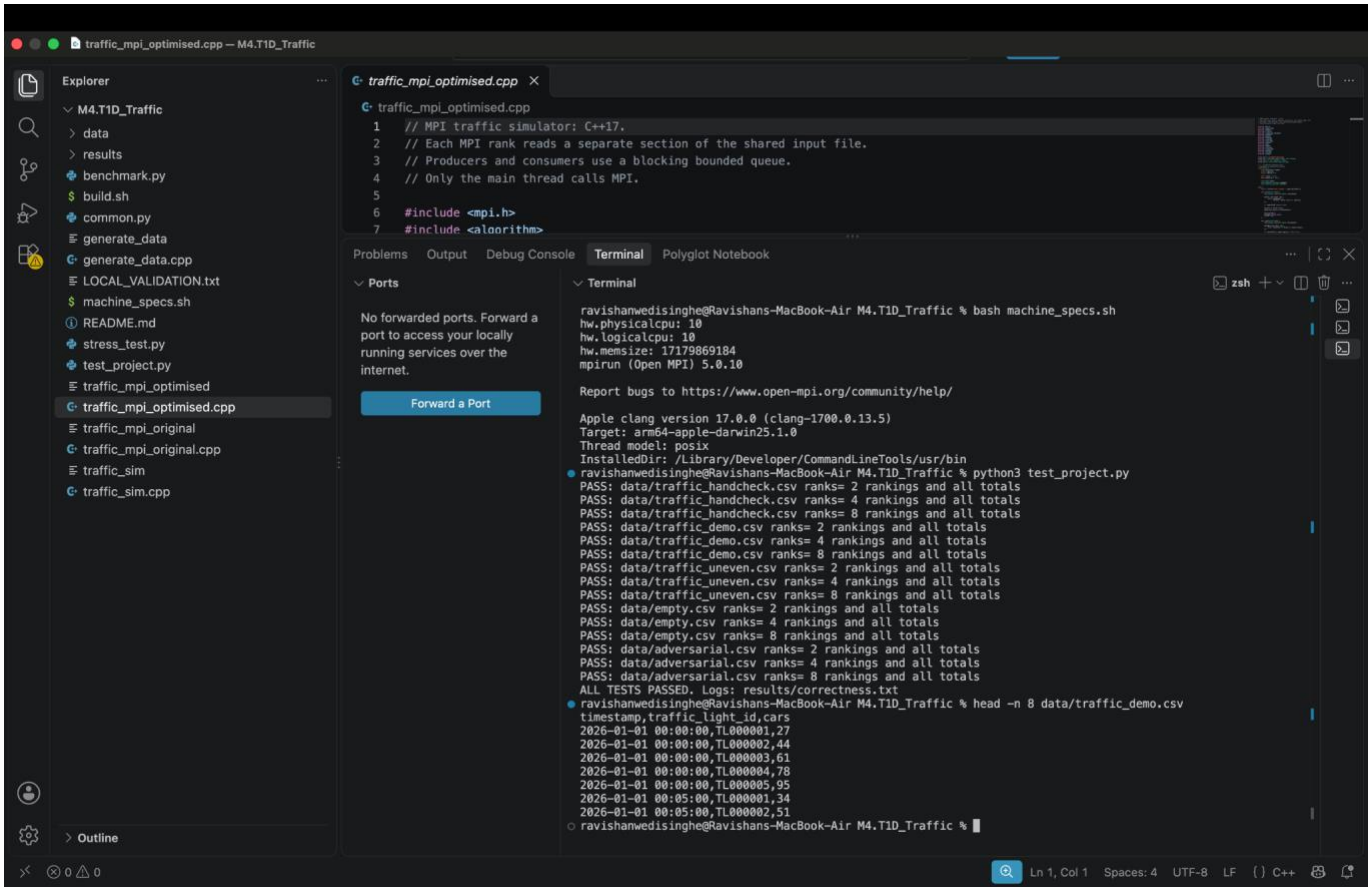
Screenshot captured on 13 October 2026 at 12:30:58 AM.



The terminal was prepared again for explanation. Build success, machine details, and the full set of PASS lines are visible. This shows the programs can be rebuilt and their tests repeated on the Mac. The source file is open above the terminal. The build script contains the exact compiler commands described in the report.

Figure 13 Tests and sample input

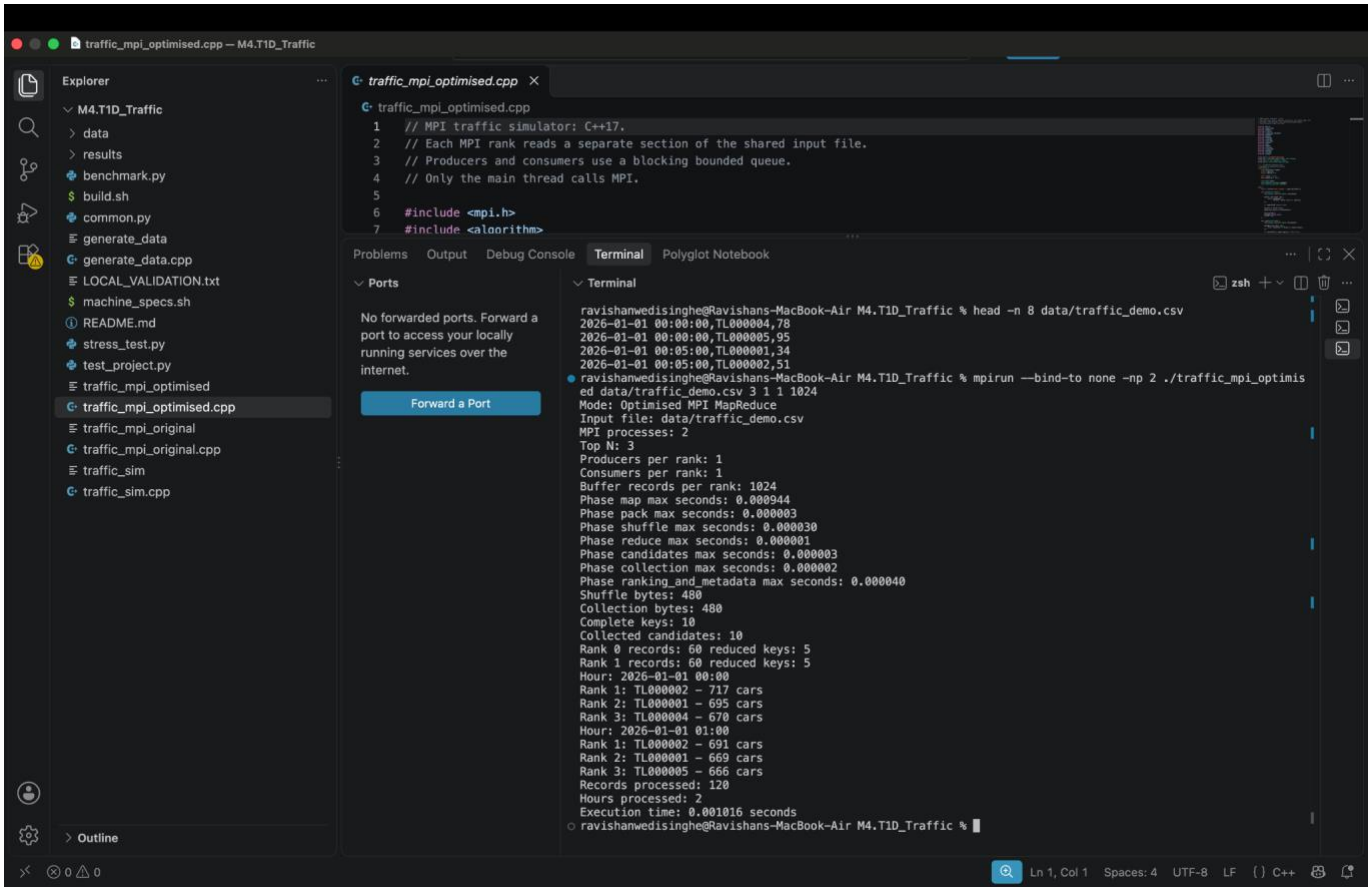
Screenshot captured on 13 October 2026 at 12:31:21 AM.



The final PASS lines are followed by head -n 8 on traffic_demo.csv. The header and sample measurements are visible. This connects the shown results to the named input file. The example has one measurement at 00:00 and the next slot at 00:05, consistent with five minute intervals.

Figure 14 Two rank demo

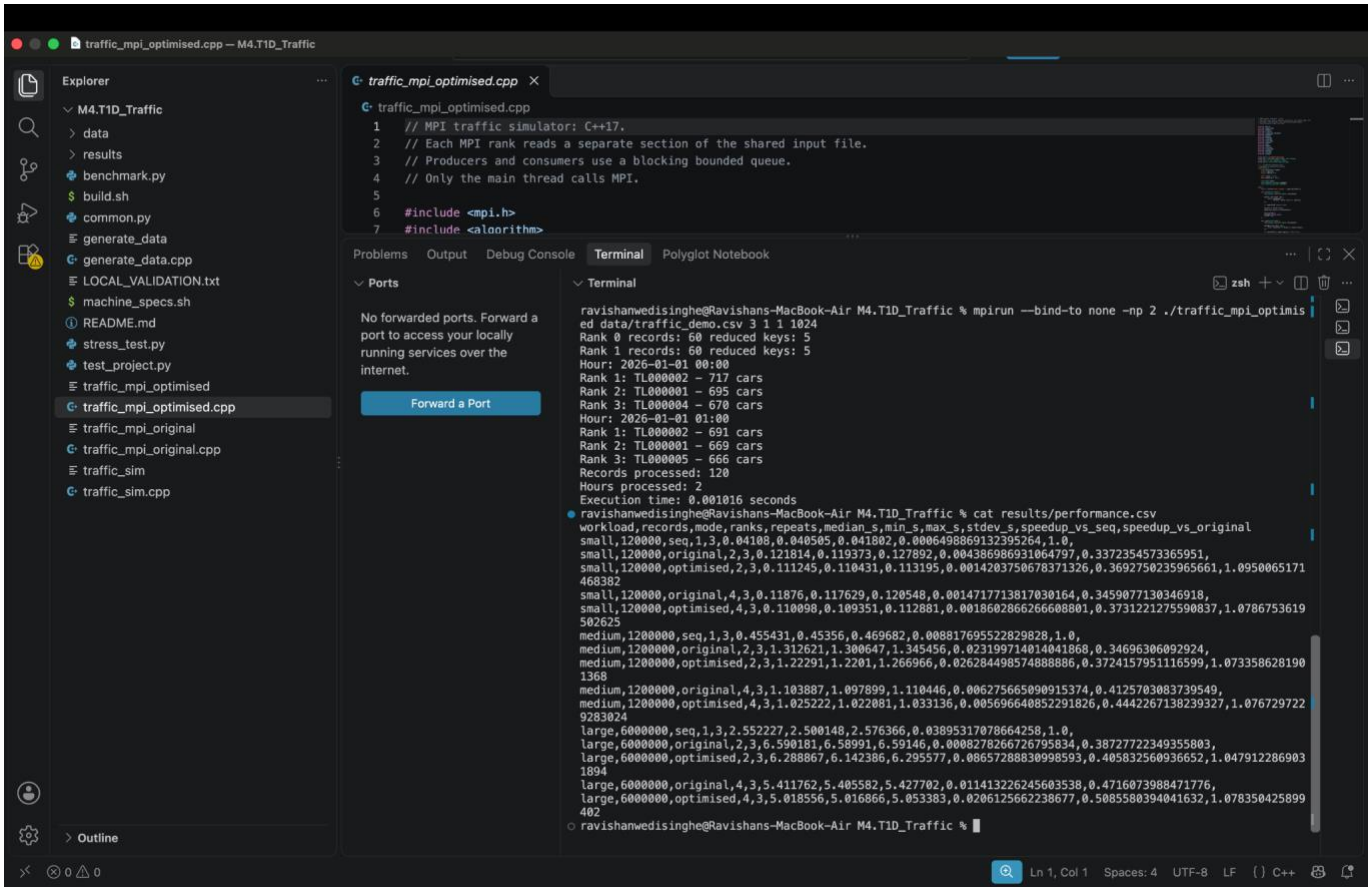
Screenshot captured on 13 October 2026 at 12:31:30 AM.



The improved demo uses two ranks and returns the same hourly top three totals. Each rank processes 60 records. The output reports 120 records and two hours. This later run takes 0.001016 seconds. The slight time change from the first demo is normal run variation; it does not change the result or replace benchmark medians.

Figure 15 Performance comparison

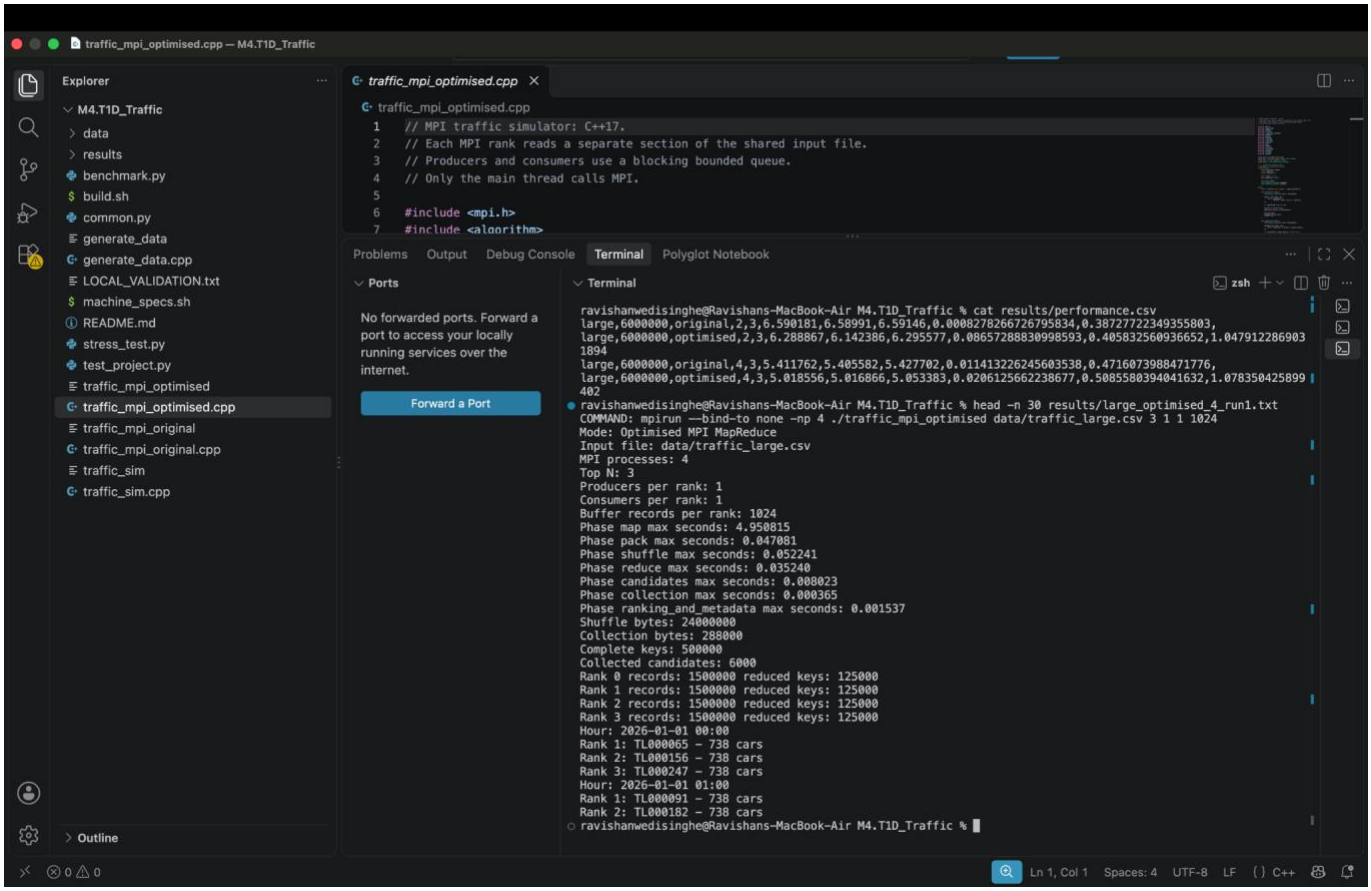
Screenshot captured on 13 October 2026 at 12:31:39 AM.



The demo output is followed by the saved performance.csv. The same median values appear, including 5.411762 seconds for original four rank large processing and 5.018556 seconds for the improved version. This is a display of earlier measurements. It does not mean the benchmark was rerun during this terminal preparation.

Figure 16 Stage times and data counts

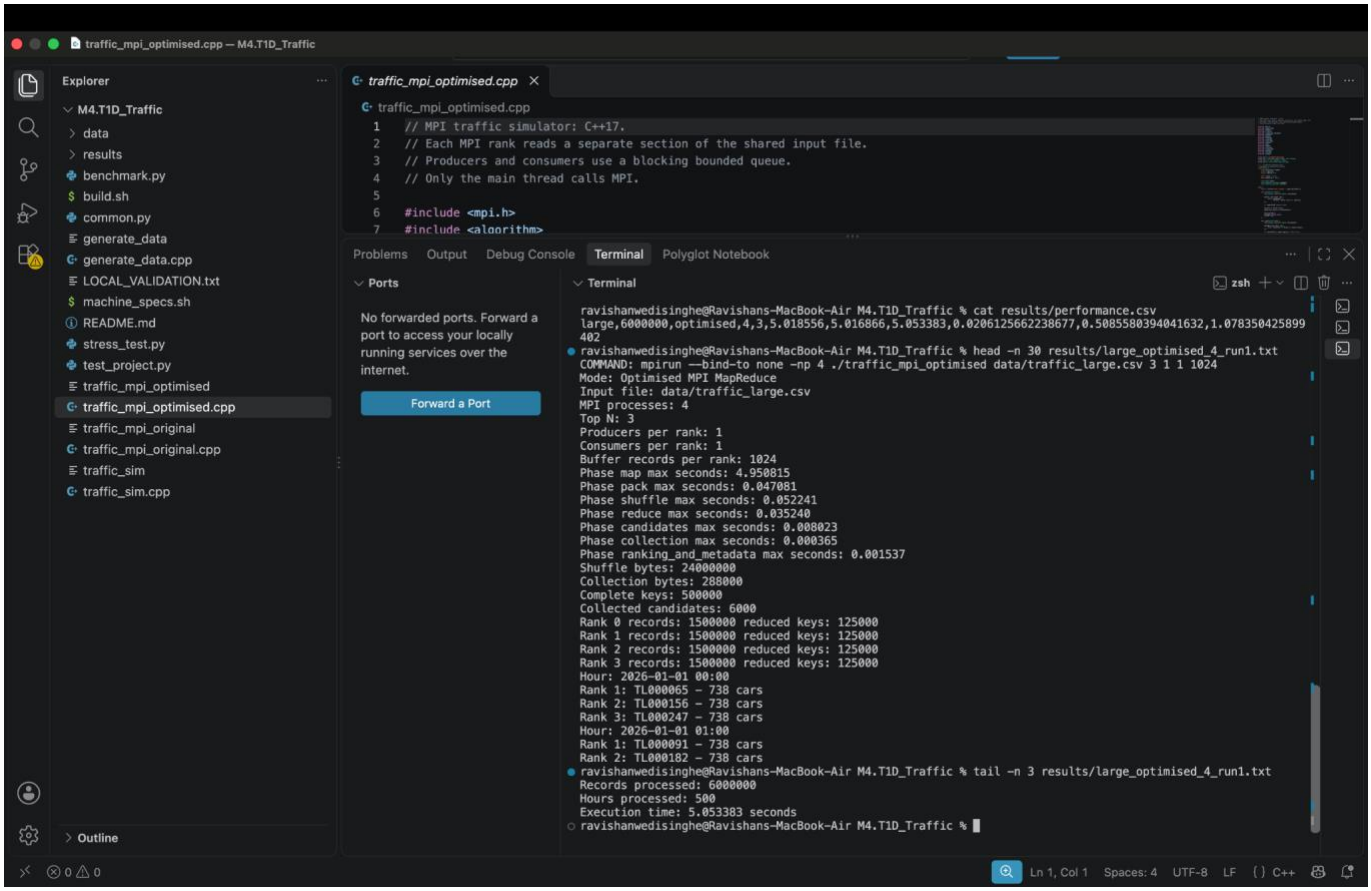
Screenshot captured on 13 October 2026 at 12:32:25 AM.



The saved large log shows the exact command, four ranks, top three, one producer and one consumer per rank, and capacity 1024. The stage timings and candidate counters are fully visible. Reading this log does not execute the program again. It is suitable evidence for explaining how the new candidate selection changes final collection.

Figure 17 Large test completion

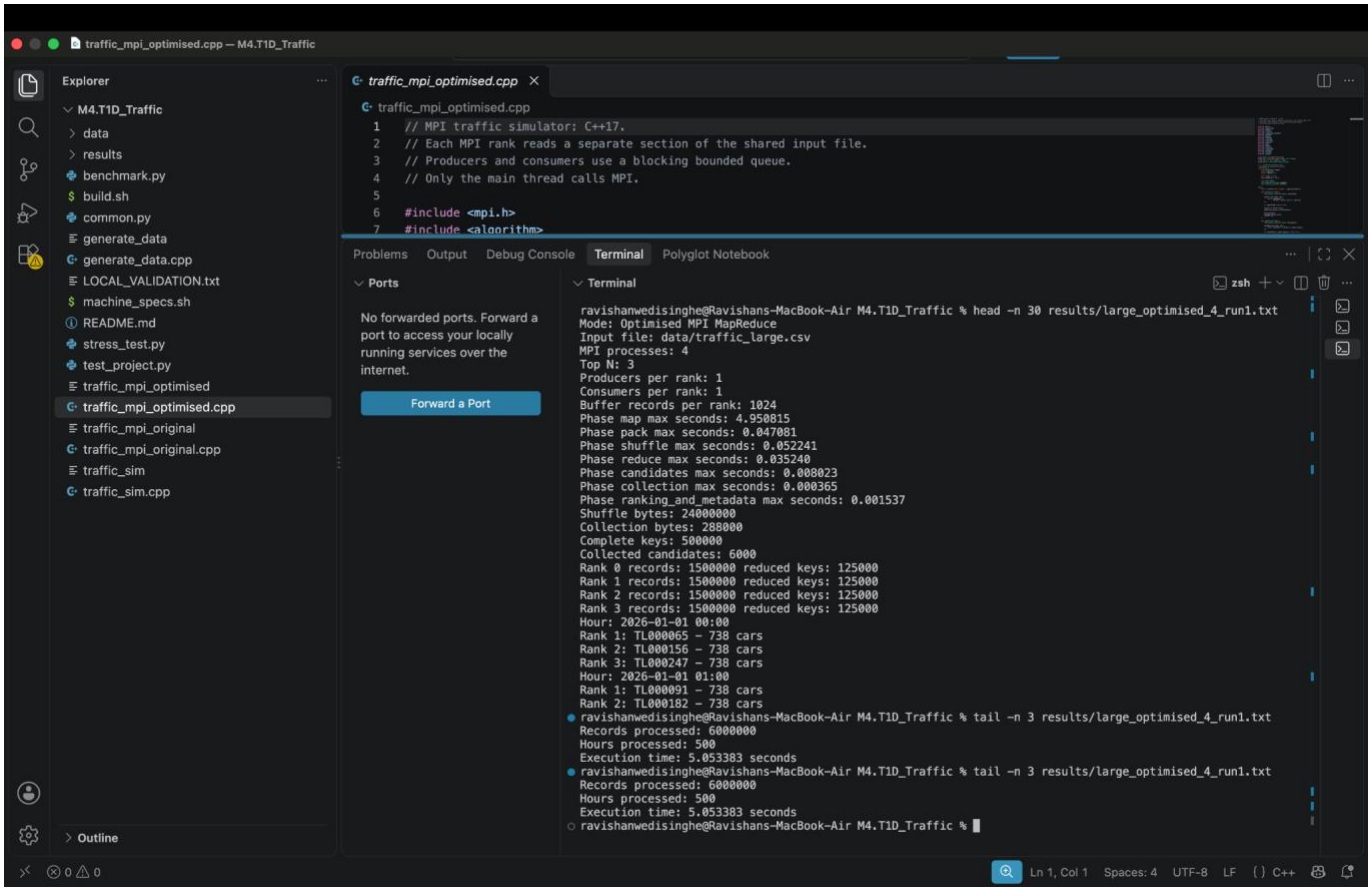
Screenshot captured on 13 October 2026 at 12:32:35 AM.



The head of the saved log is followed by its last three lines. Those lines confirm 6000000 processed records, 500 hours, and run 1 time 5.053383 seconds. Showing the beginning and end avoids scrolling through every hour while keeping the settings and completion count visible.

Figure 18 Large result shown again

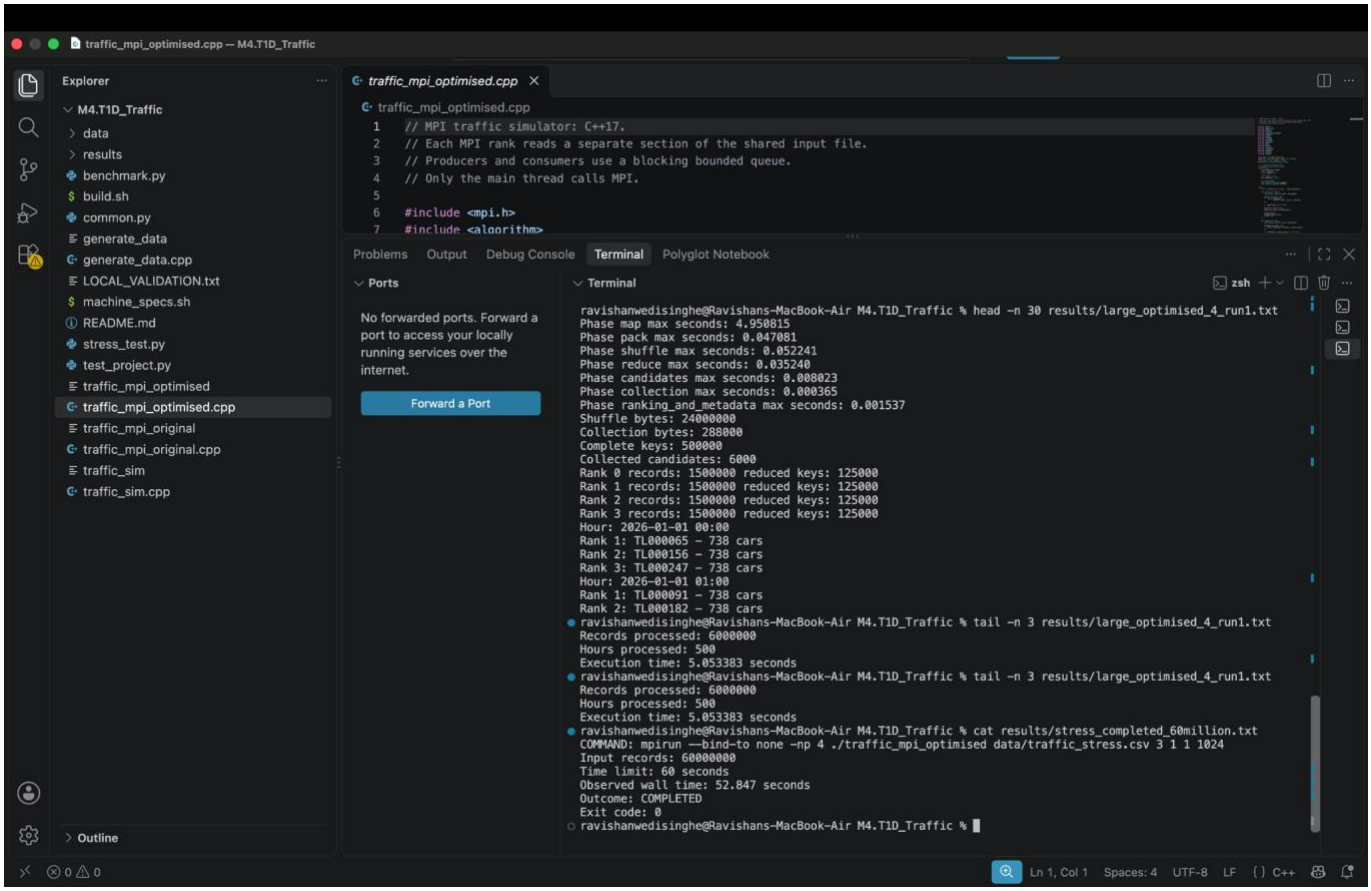
Screenshot captured on 13 October 2026 at 12:32:53 AM.



This is another view of the same stage and counter output, followed by two tail commands on the same file. Both show the same recorded time and counts. These are repeated displays of one benchmark run, not two new measurements. The repeated screenshot is retained to keep the evidence collection complete.

Figure 19 Successful stress result

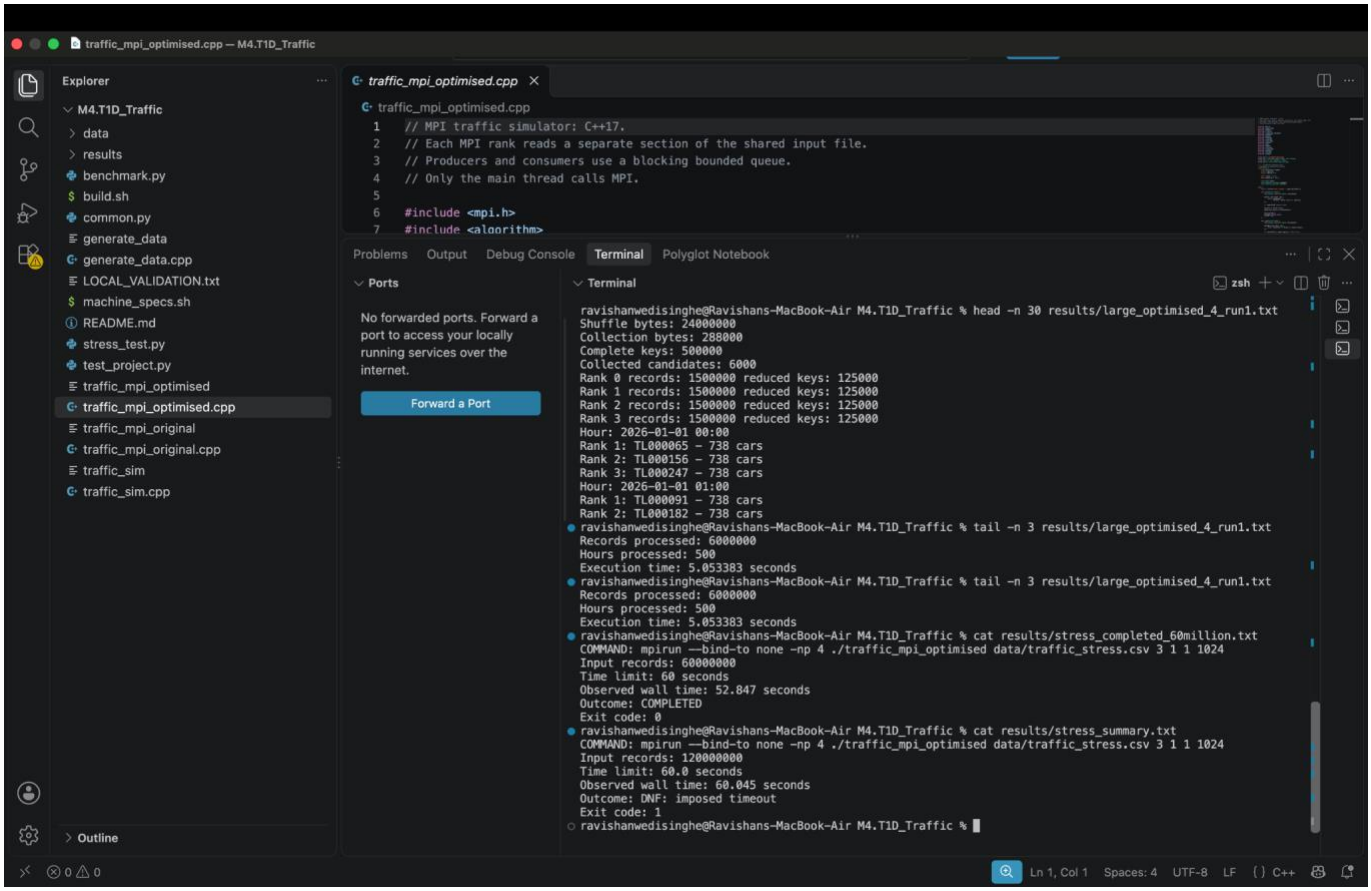
Screenshot captured on 13 October 2026 at 12:33:12 AM.



The saved large log is followed by stress_completed_60million.txt. The latter describes the earlier successful 60000000 record run and 52.847 second wall time. It preserves the successful case after stress_summary.txt was replaced by the later timeout result. The stress CSV path was reused, so the current file now refers to the later larger generated dataset.

Figure 20 Both stress results

Screenshot captured on 13 October 2026 at 12:33:23 AM.



The successful stress summary is displayed, followed by a command to show the current stress summary. The second summary records the 120000000 record run, the 60 second limit, and DNF caused by an imposed timeout. Figure 9 also preserves the full test command and outcome. This screenshot shows the terminal order for explaining both saved stress results.

Video of the Solution Running - https://deakin.au.panopto.com/Panopto/Pages/Viewer.aspx?id=3de1bb93-f5c0-4bcc-8f51-b4e200a3892a

Video demonstration - https://deakin.au.panopto.com/Panopto/Pages/Viewer.aspx?id=5dcbd01f-c585-41b0-860d-b4e10172499d

Git Hub Link - https://github.com/ravishanwedisnghe/-Task-M4.T1D